Week 2: Directed Acyclic Graphs (DAGs)

Davood Tofighi, Ph.D.

R Packages Required

```
pacman::p_load(
    "dagitty",
    "ggplot2",
    "ggdag",
    "tidyverse",
    "bnlearn",
    "Rgraphviz",
    "ggpubr"
)

ggplot2::theme_set(ggpubr::theme_pubr())
```

Chapter Overview

Causal graphs, particularly Directed Acyclic Graphs (DAGs), are fundamental tools in causal inference that help us visualize, understand, and analyze the causal relationships between variables. This comprehensive chapter will equip you with the essential knowledge to construct, interpret, and use DAGs for identifying valid causal inference strategies in applied research.

Learning Objectives:

- Understand the formal structure and terminology of causal graphs
- Distinguish between Bayesian networks and causal DAGs
- Master the three fundamental building blocks: chains, forks, and colliders
- Apply d-separation rules to determine conditional independence
- Identify valid adjustment sets for causal inference
- Recognize and avoid common pitfalls in causal graph construction

3.1 Graph Terminology

Comprehensive Explanation

A **graph** is a mathematical structure consisting of nodes (vertices) connected by edges (links). In causal inference, graphs serve as visual representations of the assumed causal relationships between variables in our study.

Fundamental Definitions:

- **Node/Vertex (V)**: Represents a variable in our causal system (e.g., treatment, outcome, confounder)
- **Edge/Arc (E)**: Represents a relationship between two variables
- **Directed Edge**: An arrow showing the direction of causation (A → B means A causes B)
- **Undirected Edge**: A line without arrows (A — B, represents association without implied causation)
- **Path**: A sequence of connected edges between nodes, regardless of direction
- **Directed Path**: A path where all edges point in the same direction
- **Cycle**: A path that starts and ends at the same node
- **Directed Acyclic Graph (DAG)**: A directed graph with no directed cycles

Additional Graph Properties:

- **Parent**: A node that has a directed edge pointing to another node
- **Child**: A node that receives a directed edge from another node
- **Ancestor**: Any node that can reach another via a directed path
- **Descendant**: Any node that can be reached from another via a directed path
- **Adjacent**: Two nodes connected by an edge
- **Degree**: The number of edges connected to a node

Intuitive Clarification

Think of a DAG as a **"causal blueprint"** of your research domain. Just as an architect's blueprint shows how rooms connect and how traffic flows through a building, a DAG shows how causal influence flows between variables in your study.

The **"acyclic" requirement** is crucial—it means we can't have causal loops. If smoking causes lung cancer, then lung cancer cannot cause smoking (at least not in the same time frame). This prevents logical contradictions and ensures we can order events causally.

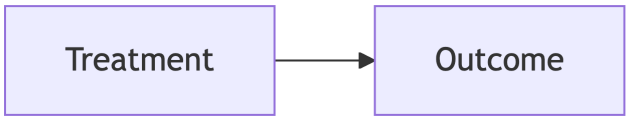
Mental Model: River Network

Imagine a river system where water flows from mountain springs to the ocean. Each spring is a "root cause," tributaries are "intermediate variables," and the ocean is your "outcome." Water (causal influence) can only flow downhill (forward in time), never uphill (backward in time).

Detailed Examples

Example 1: Simple Treatment-Outcome Relationship

```
graph LR
    T[Treatment] --> O[Outcome]
```



This represents the simplest possible causal relationship: treatment directly causes outcome.

Example 2: Confounded Relationship

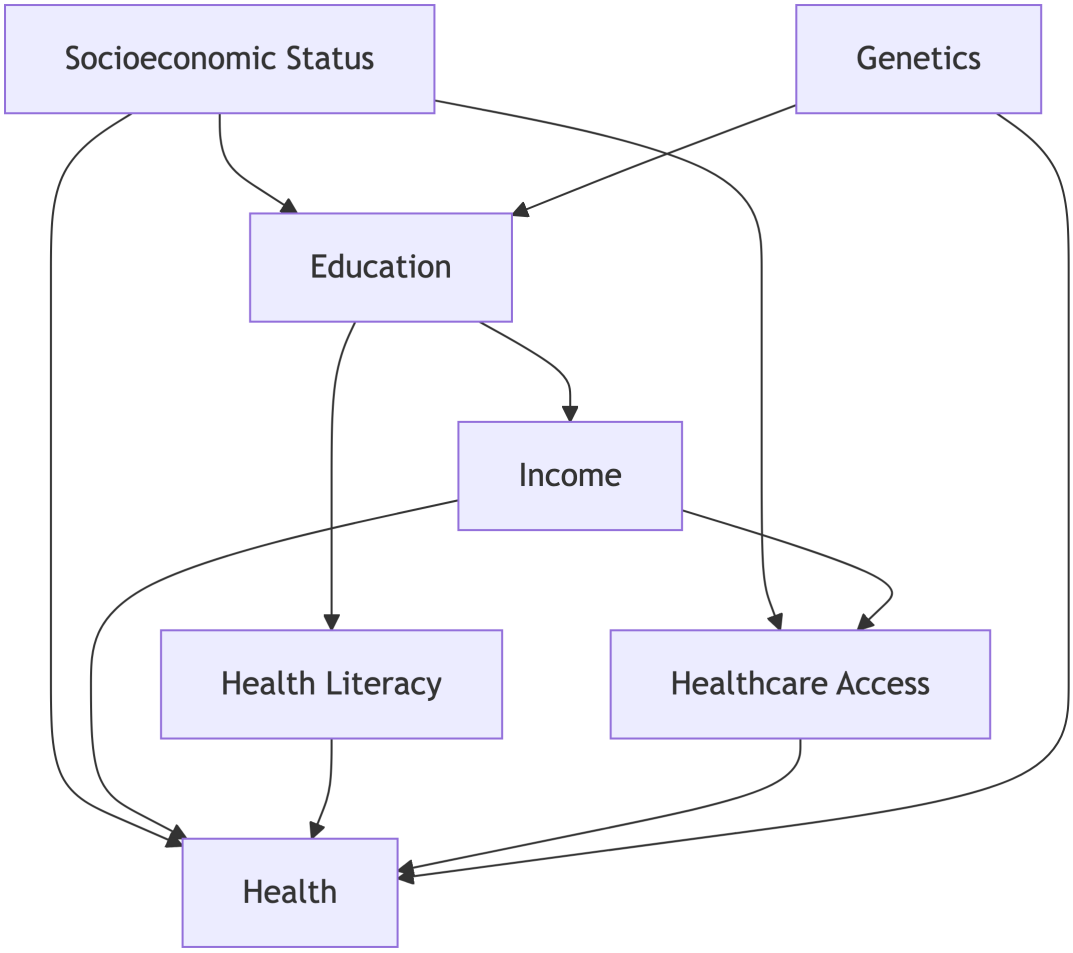
```
graph LR
    C[Confounder] --> T[Treatment]
    C --> O[Outcome]
    T --> O
```

Here, a confounder affects both treatment and outcome, creating a "backdoor path" that could bias our estimate of the treatment effect.

Example 3: Complex Research Scenario

Consider studying whether **education** improves **health outcomes**:

```
graph TD
    SES[Socioeconomic Status] --> E[Education]
    SES --> H[Health]
    SES --> HC[Healthcare Access]
    E --> I[Income]
    E --> HL[Health Literacy]
    I --> HC
    I --> H
    HL --> H
    HC --> H
    G[Genetics] --> E
    G --> H
```



This DAG shows multiple pathways through which education might affect health, plus confounding factors like socioeconomic status and genetics.

R Code: Basic Graph Operations

```
# Create a simple DAG

simple_dag <- dagitty(
    "dag {
  Treatment → Outcome
  Confounder → Treatment
  Confounder → Outcome
```

```
}"
)

# Basic graph properties
cat("Variables in DAG:", names(simple_dag), "\n")
```

```
Variables in DAG: Confounder Outcome Treatment
```

```
cat("Number of nodes:", length(names(simple_dag)), "\n")
```

```
Number of nodes: 3
```

```
print(dagitty::edges(simple_dag))
```

```
           v         w  e  x  y
1 Confounder   Outcome -> NA NA
2 Confounder Treatment -> NA NA
3  Treatment   Outcome -> NA NA
```

```
# Visualize the DAG
ggdag(simple_dag)

# Check if graph is acyclic
cat("Is DAG acyclic?", isAcyclic(simple_dag), "\n")
```

```
Is DAG acyclic? TRUE
```

```
# Find parents and children
cat(
    "Parents of Outcome:",
    toString(dagitty::parents(simple_dag, "Outcome")),
    "\n"
)
```

```
Parents of Outcome: Confounder, Treatment
```

```r
cat(
    "Children of Confounder:",
    toString(dagitty::children(simple_dag, "Confounder")),
    "\n"
)
```

```
Children of Confounder: Outcome, Treatment
```

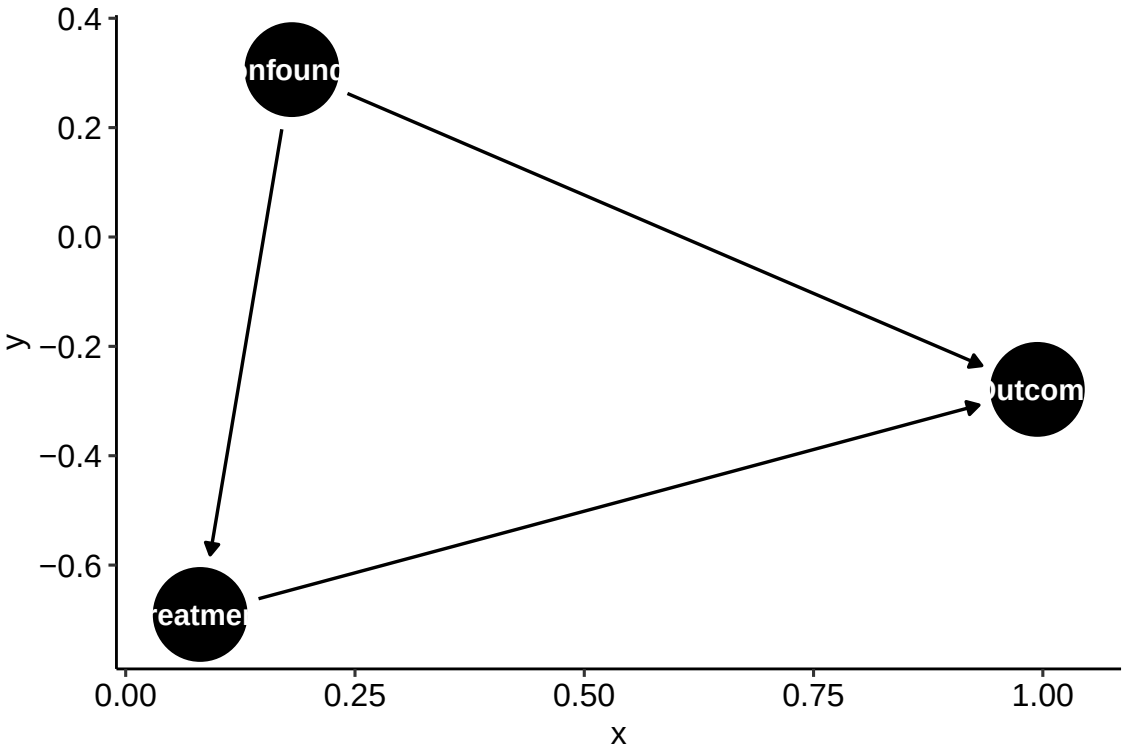


Figure 1

> **Code Explanation**
>
> - `dagitty()` creates a DAG from text specification
> - `ggdag()` provides beautiful visualizations using ggplot2
> - `parents()` and `children()` help identify direct causal relationships
> - `isAcyclic()` verifies that our graph has no cycles

3.2 Bayesian Networks

Comprehensive Explanation

A **Bayesian Network** (also called a probabilistic graphical model) is a directed acyclic graph that represents the joint probability distribution of a set of random variables. Each node represents a variable, and the graph structure encodes conditional independence assumptions.

Key Mathematical Properties:

1. **Factorization**: The joint probability distribution factors according to the graph:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^{n} P(X_i | \text{Parents}(X_i))$$

2. **Local Markov Property**: Each variable is conditionally independent of its non-descendants given its parents:

$$X_i \perp \text{NonDescendants}(X_i) | \text{Parents}(X_i)$$

3. **Global Markov Property**: Any two sets of nodes are conditionally independent given a separating set if they are d-separated by that set.

Bayesian Network vs. Causal DAG:

Aspect	Bayesian Network	Causal DAG
Purpose	Model joint probability	Represent causal relationships
Edge meaning	Statistical dependence	Direct causal effect
Interpretation	"X predicts Y given parents"	"X causes Y"
Data requirements	Can be learned from data	Requires domain knowledge
Intervention	No intervention calculus	Supports do-calculus

Intuitive Clarification

A Bayesian Network is like a **"probability assembly line"**. To compute the probability of any complex event, you break it down into simpler conditional probabilities and multiply them together according to the network structure.

Think of it this way: instead of trying to estimate the probability of rain, clouds, and wet grass all happening together (which would require enormous amounts of data), you estimate:

- P(rain) - the base rate of rain
- P(clouds | rain) - how often it's cloudy when it rains

- P(wet grass | rain, clouds) - how often grass is wet given rain and cloud conditions

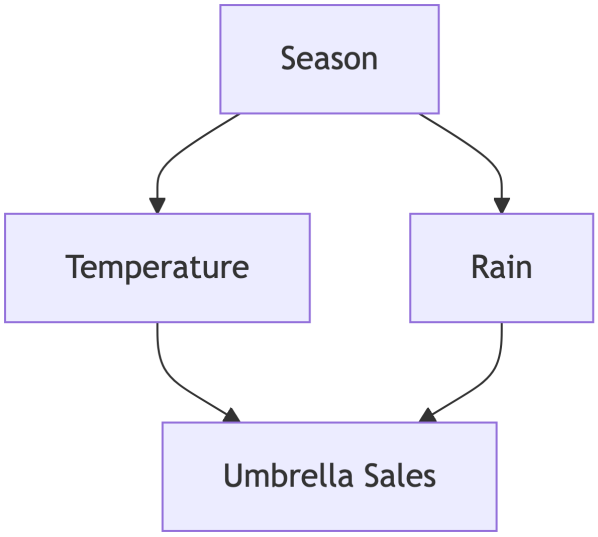
This decomposition makes complex probability calculations tractable.

> ❗ Critical Distinction
>
> While both Bayesian Networks and Causal DAGs use the same graphical structure, they make different claims:
>
> - **Bayesian Network**: "These variables are probabilistically related in this pattern"
> - **Causal DAG**: "These variables have cause-and-effect relationships in this pattern"
>
> Only Causal DAGs support **interventional reasoning** (what happens if we change X?).

Detailed Example: Weather Prediction Network

Consider a weather prediction system with four variables:

```
graph TD
    S[Season] --> T[Temperature]
    S --> R[Rain]
    T --> U[Umbrella Sales]
    R --> U
```



Bayesian Network Factorization:

$$P(S, T, R, U) = P(S) \cdot P(T|S) \cdot P(R|S) \cdot P(U|T, R)$$

Conditional Independence Relationships:

- Temperature $\perp$ Rain | Season (given the season, temperature and rain are independent)
- Season $\perp$ Umbrella Sales | Temperature, Rain (given temp and rain, season doesn't add info about umbrella sales)

R Code: Bayesian Network Analysis

```r
# Simulate Data following Our Weather Network Structure
set.seed(123)
n <- 1000

# Generate Data according to Our Assumed Structure
season <- sample(c("Spring", "Summer", "Fall", "Winter"), n, replace = TRUE)
temp <- ifelse(
    season %in% c("Summer", "Spring"),
    rnorm(n, 75, 10),
    rnorm(n, 45, 15)
)
```

```
rain_prob <- ifelse(season %in% c("Spring", "Fall"), 0.6, 0.2)
rain <- rbinom(n, 1, rain_prob)
umbrella_sales <- rpois(n, 10 + 0.1 * temp + 5 * rain)

# Create Data Frame
weather_data <- data.frame(
    Season = factor(season),
    Temperature = temp,
    Rain = factor(rain),
    Umbrella_Sales = umbrella_sales
)
weather_data$Umbrella_Sales <- as.numeric(weather_data$Umbrella_Sales)
# Learn Bayesian Network Structure from Data
bn_structure <- hc(weather_data) # Hill-climbing algorithm
plot(bn_structure)

# Fit Parameters to the Learned Structure
bn_fitted <- bn.fit(bn_structure, weather_data)

# Query the Network: What's P(Rain = 1 | Season = "Spring")?
cpquery(bn_fitted, (Rain == "1"), (Season == "Spring"))
```

```
[1] 0.584375
```

```
# More Complex Query: P(Umbrella_Sales > 15 | Temperature > 70, Rain = 1)
cpquery(bn_fitted, (Umbrella_Sales > 15), (Temperature > 70 & Rain == "1"))
```

```
[1] 0.9481383
```

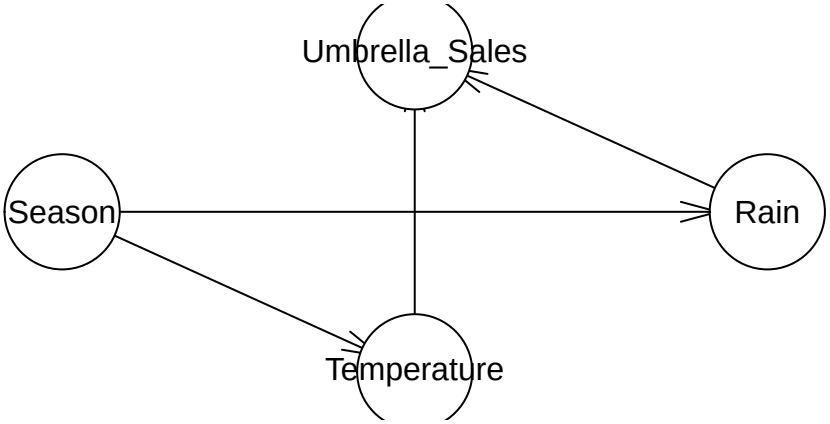


Figure 2: Code for Bayesian Network Analysis

Code Explanation

- `hc()` learns network structure using hill-climbing algorithm
- `bn.fit()` estimates conditional probability tables from data
- `cpquery()` performs conditional probability queries
- Notice how we can ask probabilistic questions but not causal questions ("what if we change the season?")

3.3 Causal Graphs

Comprehensive Explanation

A **Causal Graph** (or Causal DAG) is a directed acyclic graph where nodes represent variables and directed edges represent **direct causal relationships**. Unlike Bayesian Networks, Causal DAGs make strong claims about cause-and-effect relationships in the real world.

Fundamental Principles:

1. **Causal Edges**: X → Y means X is a direct cause of Y (X causally influences Y, holding all other variables constant)
2. **Missing Edges**: The absence of X → Y means X is not a direct cause of Y (though X might still affect Y through other variables)
3. **Temporal Ordering**: Causes must precede effects in time (though the DAG itself is atemporal)

Critical Assumptions for Causal DAGs:

1. **Causal Markov Assumption**: Each variable is independent of its non-descendants conditional on its direct causes

$$X \perp \text{NonDescendants}(X) | \text{Parents}(X)$$

2. **Faithfulness/Stability**: The probability distribution contains all and only the independence relationships implied by the DAG structure
3. **Causal Sufficiency**: All common causes of included variables are either included in the DAG or explicitly represented as unobserved confounders
4. **No Selection Bias**: The sample is representative of the target population (or selection mechanisms are properly modeled)

Intuitive Clarification

A Causal DAG is your **"theory of how the world works"** made explicit and testable. It forces you to articulate your beliefs about: - What causes what? - What doesn't cause what? - What are the potential confounders? - What are the mechanisms (mediators)?

Unlike statistical models that can be "discovered" from data, causal relationships must be specified based on **substantive knowledge**—theory, previous research, expert opinion, and logical reasoning.

⚠ Fundamental Limitation

Correlation does not imply causation, and no amount of statistical analysis can establish causal relationships from observational data alone. Your DAG represents your causal assumptions—these must come from outside the data.

DAG Construction Strategy

1. **Start with the research question**: What is your treatment? What is your outcome?
2. **Brainstorm all relevant variables**: What could affect the treatment? The outcome? Both?
3. **Consider temporal ordering**: What comes first, second, third?
4. **Think about mechanisms**: How might the treatment affect the outcome?
5. **Identify potential confounders**: What common causes create backdoor paths?
6. **Be explicit about what you're assuming**: Every missing edge is also an assumption!

Detailed Example: Education and Health Outcomes

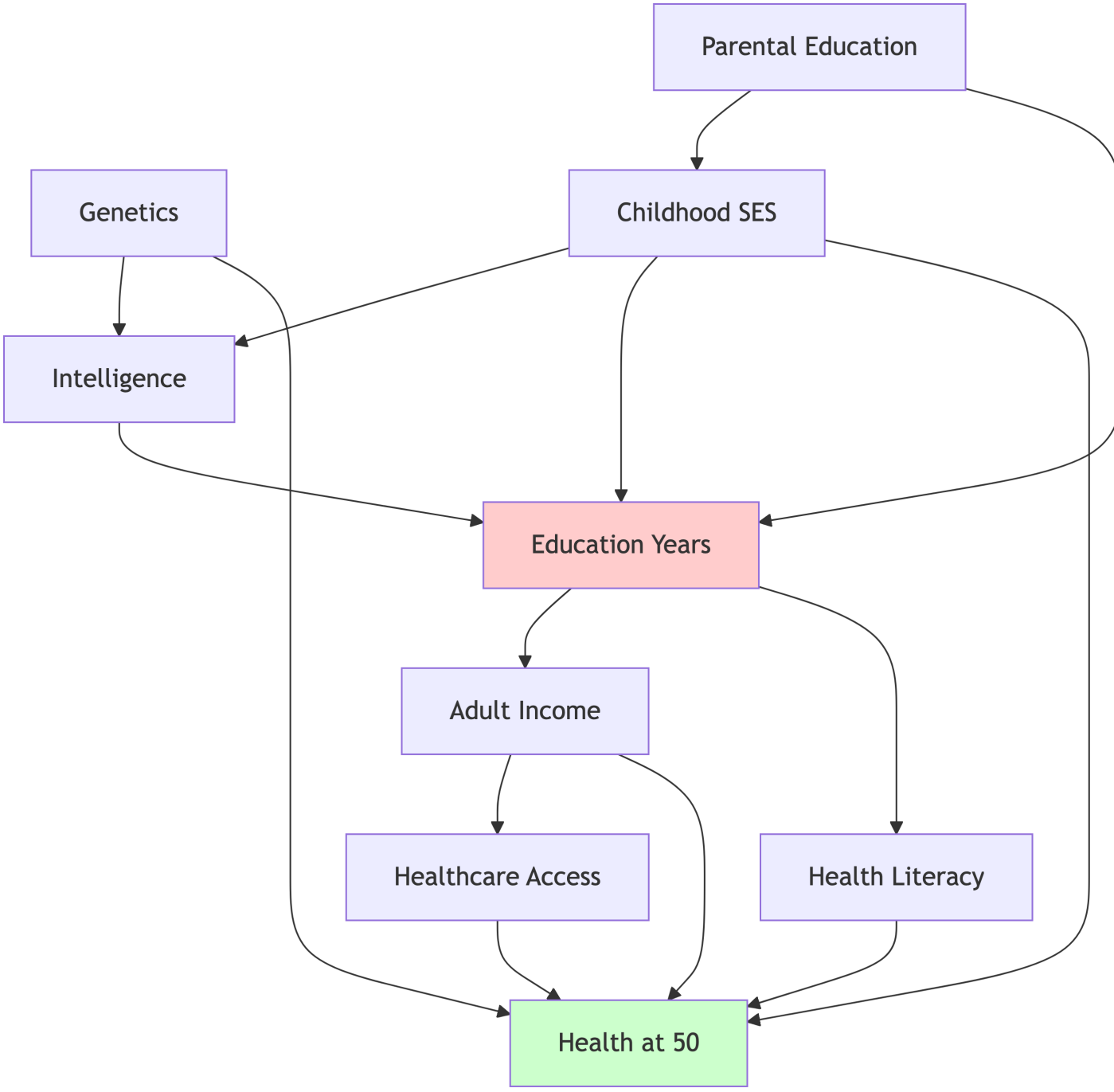
Research Question: Does higher education improve health outcomes in adulthood?

Variables to Consider: - **E**: Years of education (treatment) - **H**: Health outcomes at age 50 (outcome) - **SES**: Childhood socioeconomic status - **IQ**: Intelligence/cognitive ability - **I**: Adult income - **HL**: Health literacy - **HC**: Healthcare access - **G**: Genetic factors - **PE**: Parental education

Causal Theory: Education might improve health through multiple pathways: increased income (affording better healthcare), health literacy (better health decisions), and social networks (social support). However, this relationship is confounded by childhood SES, intelligence, and genetics.

```
graph TD
    G[Genetics] --> IQ[Intelligence]
    G --> H[Health at 50]
    PE[Parental Education] --> SES[Childhood SES]
    PE --> E[Education Years]
    SES --> IQ
    SES --> E
    SES --> H
    IQ --> E
    E --> I[Adult Income]
    E --> HL[Health Literacy]
    I --> HC[Healthcare Access]
    HL --> H
    HC --> H
    I --> H

    style E fill:#ffcccc
    style H fill:#ccffcc
```



Causal Pathways Analysis:

- **Direct effects**: E → I → H, E → HL → H
- **Indirect effects**: E → I → HC → H

- **Confounding paths**: E ← SES → H, E ← IQ ← G → H, E ← PE → SES → H

R Code: Causal DAG Analysis

```
# Define the Education-health Causal DAG
education_dag <- dagitty::dagitty(
    "dag {
  Genetics → Intelligence → Education
  Genetics → Health
  ParentalEd → ChildhoodSES → Education
  ParentalEd → Education
  ChildhoodSES → Intelligence
  ChildhoodSES → Health
  Education → Income → Health
  Education → HealthLiteracy → Health
  Income → HealthcareAccess → Health
}"
)

# Set Coordinates for Better Visualization
dagitty::coordinates(education_dag) <- list(
    x = c(
        Genetics = 0,
        Intelligence = 1,
        ParentalEd = 0,
        ChildhoodSES = 1,
        Education = 2,
        Income = 3,
        HealthLiteracy = 3,
        HealthcareAccess = 4,
        Health = 4
    ),
    y = c(
        Genetics = 0,
        Intelligence = 0.5,
        ParentalEd = 2,
        ChildhoodSES = 1.5,
        Education = 1,
```

```
        Income = 0.5,
        HealthLiteracy = 1.5,
        HealthcareAccess = 0.5,
        Health = 1
    )
)

# Visualize the DAG
ggdag(education_dag) +
    theme_dag_blank() +
    labs(title = "Causal DAG: Education → Health")

# Identify All Paths from Education to Health
all_paths <- dagitty::paths(education_dag, "Education", "Health")
print(all_paths)
```

```
$paths
[1] "Education -> HealthLiteracy -> Health"
[2] "Education -> Income -> Health"
[3] "Education -> Income -> HealthcareAccess -> Health"
[4] "Education <- ChildhoodSES -> Health"
[5] "Education <- ChildhoodSES -> Intelligence <- Genetics -> Health"
[6] "Education <- Intelligence <- ChildhoodSES -> Health"
[7] "Education <- Intelligence <- Genetics -> Health"
[8] "Education <- ParentalEd -> ChildhoodSES -> Health"
[9] "Education <- ParentalEd -> ChildhoodSES -> Intelligence <- Genetics -> Health"

$open
[1]  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE FALSE
```

```
# Find Minimal Adjustment Sets (what to Control for)
adj_sets <- dagitty::adjustmentSets(
    education_dag,
    exposure = "Education",
    outcome = "Health"
)
print(adj_sets)
```

```
{ ChildhoodSES, Genetics }
{ ChildhoodSES, Intelligence }
```

```
# Test Specific Adjustment Sets
is_valid <- dagitty::isAdjustmentSet(
    education_dag,
    "Education",
    "Health",
    c("ChildhoodSES", "Intelligence")
)
cat("Is {ChildhoodSES, Intelligence} a valid adjustment set?", is_valid, "\n")
```

```
Is {ChildhoodSES, Intelligence} a valid adjustment set? FALSE
```

```
# What Variables Are Ancestors of the Treatment?
ancestors_treat <- dagitty::ancestors(education_dag, "Education")
cat("Ancestors of Education:", toString(ancestors_treat), "\n")
```

```
Ancestors of Education: Education, ParentalEd, Intelligence, Genetics, ChildhoodSES
```

```
# What Variables Are Descendants of the Treatment?
descendants_treat <- dagitty::descendants(education_dag, "Education")
cat("Descendants of Education:", toString(descendants_treat), "\n")
```

```
Descendants of Education: Education, Income, HealthcareAccess, Health, HealthLiteracy
```

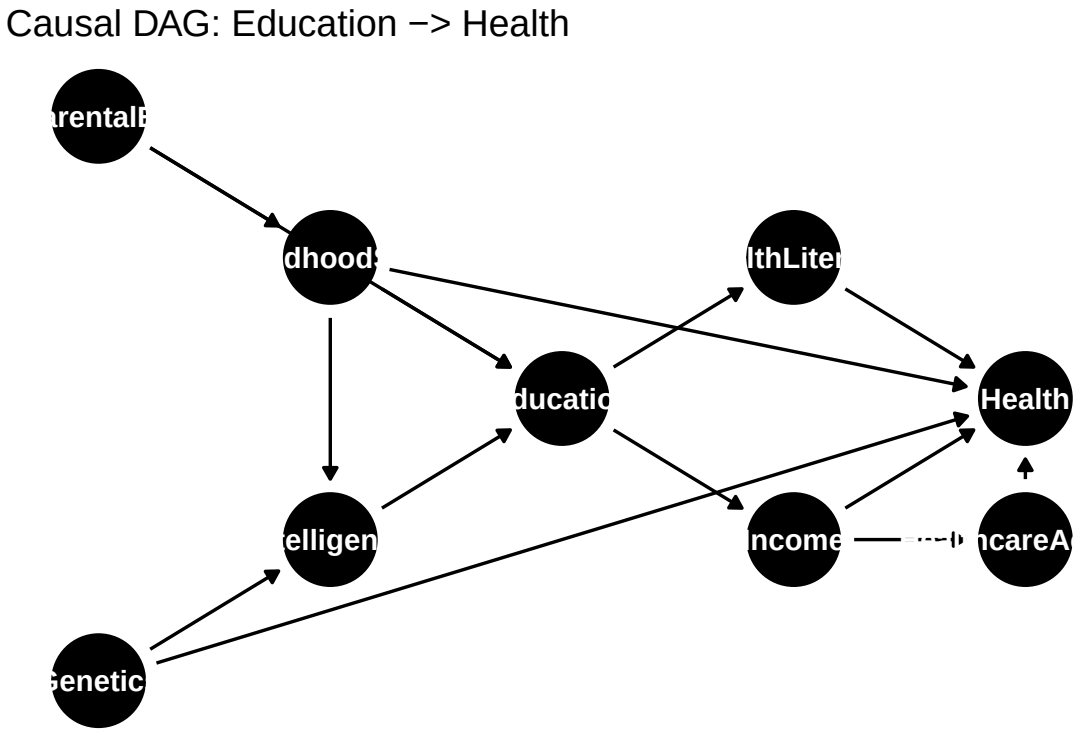


Figure 3: Code for Causal DAG Analysis

Code Interpretation

- `paths()` shows all pathways between treatment and outcome
- `adjustmentSets()` finds valid sets of variables to control for
- Notice that we need to control for confounders (ChildhoodSES, Intelligence) but NOT mediators (Income, HealthLiteracy)
- Controlling for descendants would create selection bias

3.4 Two-Node Graphs and Graphical Building Blocks

Comprehensive Explanation

Understanding causal relationships begins with the simplest possible case: **two-node graphs**. These basic structures serve as the fundamental building blocks for constructing complex causal networks.

The Three Possible Two-Node Relationships:

1. **Direct Causation**: $X \rightarrow Y$

- X directly causes Y
- Association between X and Y is flows from X to Y
- Intervening on X will change Y

2. **No Direct Relationship**: $X \quad Y$ (no edge)
   - X does not directly cause Y (and vice versa)
   - Any observed association is spurious or mediated through other variables
   - Intervening on X alone will not change Y
3. **Bidirectional Causation**: Not allowed in DAGs
   - Would create a cycle: $X \leftrightarrow Y$
   - Must be resolved by choosing direction or including temporal subscripts
   - Often indicates missing variables or feedback loops

Important Considerations:

- **Temporal Order**: In $X \rightarrow Y$, X must precede or be simultaneous with Y
- **Mechanism**: There should be a plausible causal mechanism linking X to Y
- **Confounding**: The absence of confounders is assumed in simple two-node graphs
- **Magnitude**: The strength of the arrow is not represented in the DAG structure

Intuitive Clarification

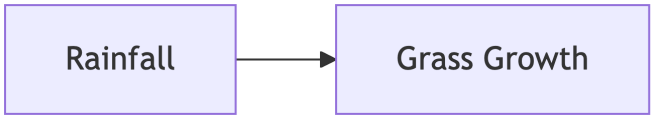
Two-node graphs are like **"causal atoms"**—the simplest indivisible units of causal reasoning. Just as complex molecules are constructed by combining simple atoms according to chemical rules, complex causal systems are built by combining these basic two-variable relationships according to causal logic.

> Building Complex DAGs
>
> Start your DAG construction by listing all pairs of variables and asking: "Does variable A directly cause variable B?" Go through every possible pair systematically. This prevents you from missing important connections or making contradictory assumptions.

Detailed Examples

Example 1: Clear Direct Causation

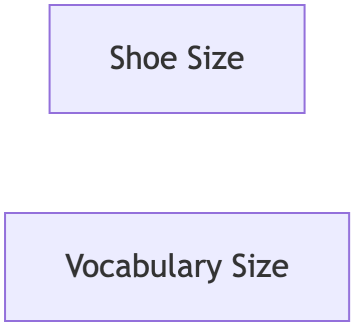
```
graph LR
    R[Rainfall] --> G[Grass Growth]
```



Justification: Rainfall directly provides water necessary for grass growth. The mechanism is clear (hydration), temporal order is correct (rain comes before growth), and there's strong biological theory supporting this relationship.

Example 2: No Direct Relationship

```
graph LR
    S[Shoe Size]
    V[Vocabulary Size]
```



Justification: Shoe size doesn't directly cause vocabulary size. Any observed correlation (e.g., in children) is explained by age as a common cause of both variables.

Example 3: Apparent Bidirectional Causation (Problematic)

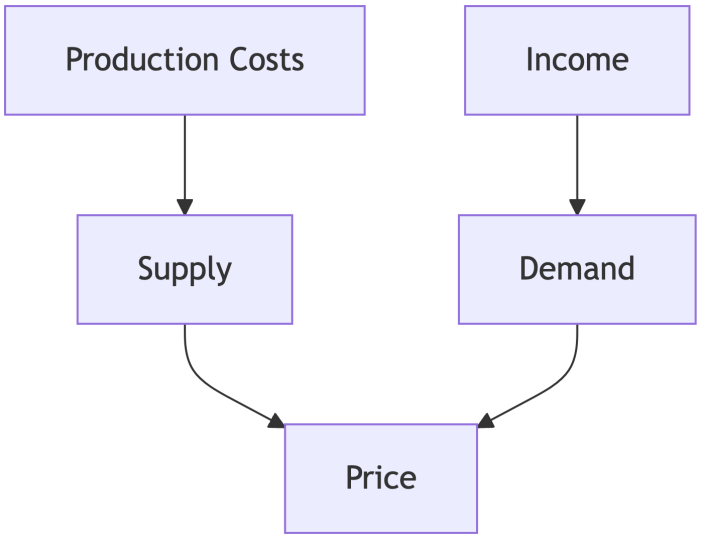
Problematic: Supply ↔ Demand (creates a cycle)

Solution 1 - Temporal Resolution:

```
graph LR
    S1[Supply t-1] --> D[Demand t]
    D --> S2[Supply t+1]
```

Solution 2 - Include Missing Variables:

```
graph TD
    PC[Production Costs] --> S[Supply]
    I[Income] --> D[Demand]
    S --> P[Price]
    D --> P
```



R Code: Two-Node Analysis

```
# Example 1: Direct Causation
direct_dag <- dagitty("dag { Rainfall → GrassGrowth }")
```

```
# Example 2: No Relationship (independent variables)
independent_dag <- dagitty("dag { ShoeSize; VocabularySize }")

# Example 3: Resolved Bidirectional Relationship
market_dag <- dagitty(
    "dag {
  ProductionCosts → Supply → Price
  Income → Demand → Price
}"
)

# Visualize All Three
par(mfrow = c(1, 3))
plot(direct_dag, main = "Direct Causation")
plot(independent_dag, main = "No Relationship")
plot(market_dag, main = "Resolved Market Dynamics")
```

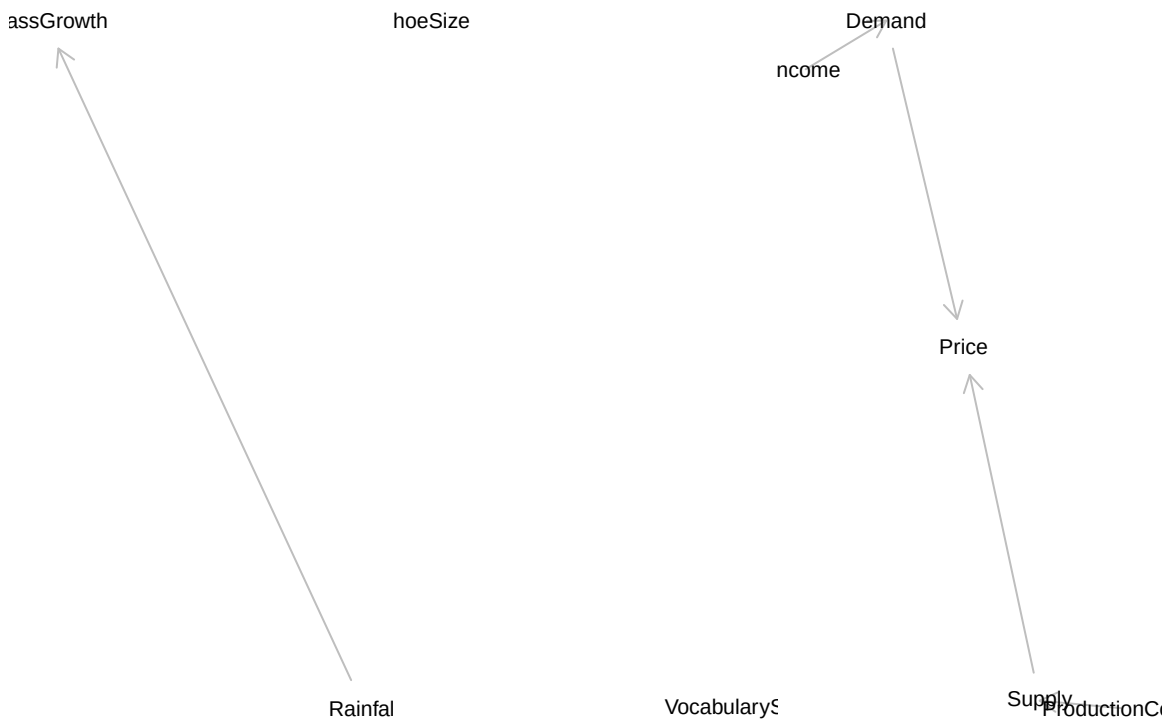


Figure 4: Three Possible Two-Node Graphs

```
# Test Implications of Each Structure
# Direct Causation: Rainfall and GrassGrowth Should Be Associated
impliedConditionalIndependencies(direct_dag)

# No Relationship: Should Be Independent
impliedConditionalIndependencies(independent_dag)
```

```
ShSz _||_ VcbS
```

```
# Market Model: More Complex Independence Relationships
impliedConditionalIndependencies(market_dag)
```

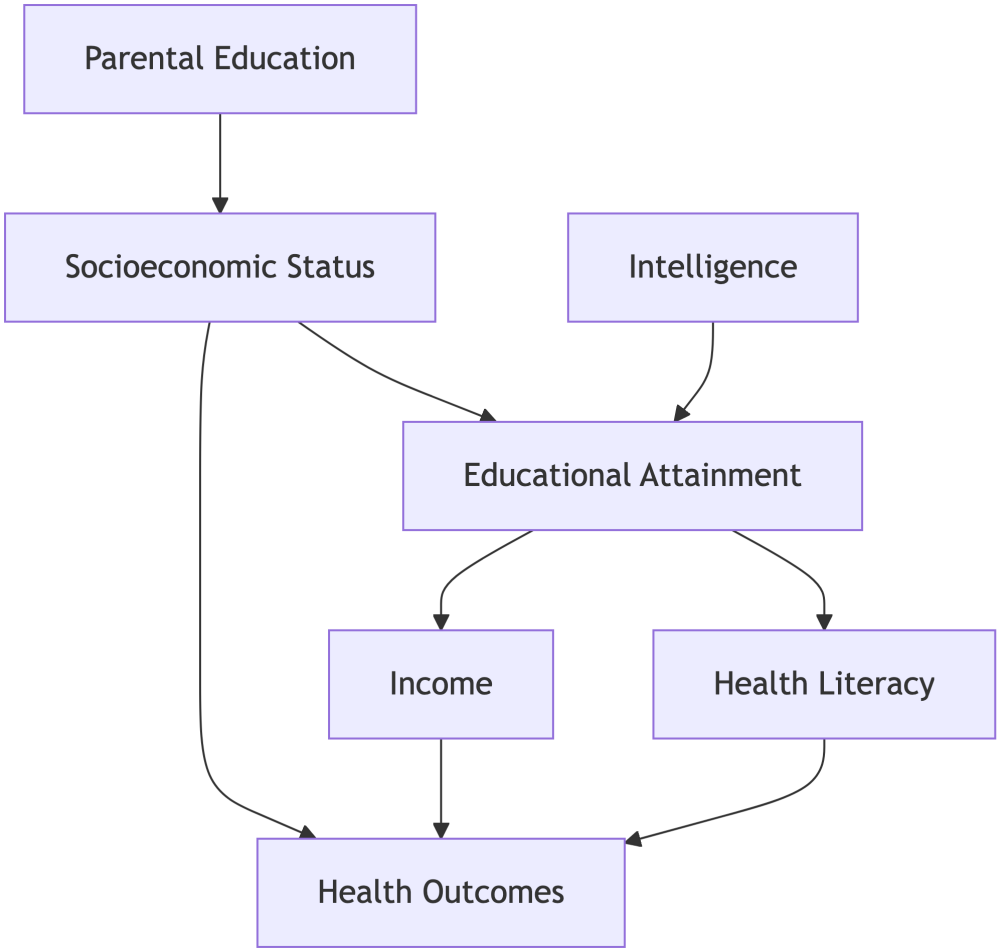
```
Dmnd _||_ PrdC
Dmnd _||_ Sppl
Incm _||_ Pric | Dmnd
Incm _||_ PrdC
Incm _||_ Sppl
Pric _||_ PrdC | Sppl
```

Building Blocks in Practice: Education Research

Individual Two-Node Relationships: - Socioeconomic Status → Educational Attainment - Educational Attainment → Income
- Parental Education → Child Education - Intelligence → Educational Attainment - Educational Attainment → Health Outcomes - Income → Health Outcomes - Educational Attainment → Health Literacy - Health Literacy → Health Outcomes

Combined into Larger DAG:

```
graph TD
    PE[Parental Education] --> SES[Socioeconomic Status]
    SES --> EA[Educational Attainment]
    SES --> HO[Health Outcomes]
    IQ[Intelligence] --> EA
    EA --> I[Income]
    EA --> HL[Health Literacy]
    I --> HO
    HL --> HO
```



3.5 Chains and Forks

Comprehensive Explanation

Chains and **Forks** are fundamental three-node structures that appear repeatedly in causal graphs and represent two distinct types of confounding and causal relationships.

Chains (Mediation Structure)

Structure: $X \to M \to Y$

Characteristics: - X causes M, and M causes Y - M is a **mediator** of the X→Y relationship - The **total effect** of X on Y flows through M - The **direct effect** of X on Y would be zero if there's no direct X→Y edge

Statistical Properties: - X and Y are **marginally associated** (unconditionally) - X and Y are **conditionally independent** given M - Conditioning on M **blocks** the causal pathway

Causal Interpretation: - **Total Effect**: How much Y changes when X changes (don't control for M) - **Direct Effect**: How much Y changes when X changes, holding M constant (control for M) - **Indirect Effect**: The effect that flows through M (Total - Direct)

Forks (Common Cause Structure)

Structure: $X \leftarrow Z \rightarrow Y$

Characteristics: - Z causes both X and Y - Z is a **common cause** or **confounder** - Creates **spurious association** between X and Y - No direct causal relationship between X and Y

Statistical Properties: - X and Y are **marginally associated** (unconditionally) - X and Y are **conditionally independent** given Z - Conditioning on Z **blocks** the spurious association

Causal Interpretation: - The X-Y association is entirely due to their common cause Z - To estimate the causal effect of X on Y, you must control for Z

Intuitive Clarification

Chain Analogy: Think of a chain as a **relay race** where the causal effect (like a baton) passes from runner to runner. If you want to know the total effect of the first runner on the finish time, don't stop the baton at the second runner. But if you want to know the direct effect of the first runner on the third runner, then you do need to "stop" at the second runner.

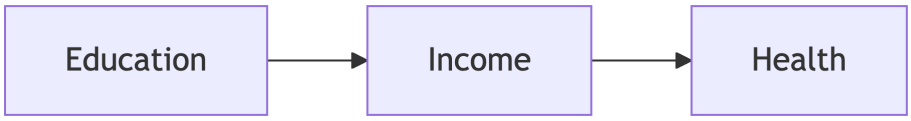
Fork Analogy: Think of a fork as a **river splitting into tributaries**. The main river (common cause) affects both branches (X and Y). If you see the two branches flowing at the same rate, it's not because one affects the other—it's because they share the same source.

> ! Critical Decision Rule
>
> - **Chain (X→M→Y)**: Control for M only if you want the **direct effect**. Don't control for M if you want the **total effect**.
> - **Fork (X←Z→Y)**: Always control for Z if you want the **causal effect** of X on Y, because Z is a confounder.

Detailed Example 1: Chain (Mediation)

Research Question: Does education improve health outcomes through increased income?

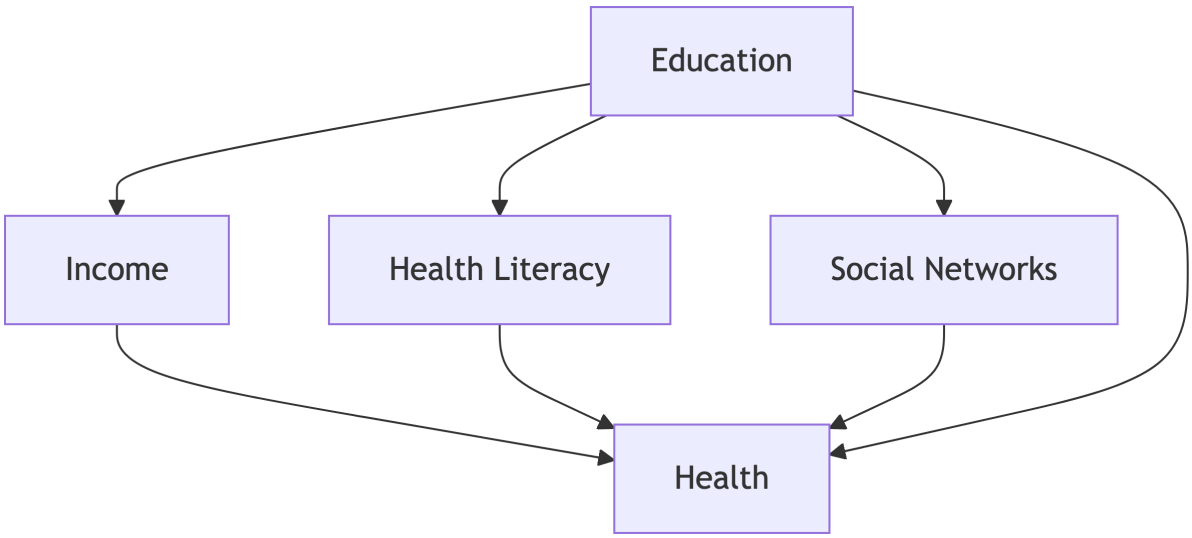
```
graph LR
    E[Education] --> I[Income] --> H[Health]
```



Causal Questions: 1. **Total effect**: How much does health improve with each additional year of education? (Don't control for income) 2. **Direct effect**: How much does health improve with education, independent of income changes? (Control for income) 3. **Mediated effect**: How much of education's effect works through income? (Total - Direct)

Real-world complexity: Education might also have direct effects on health through health literacy, social networks, etc.

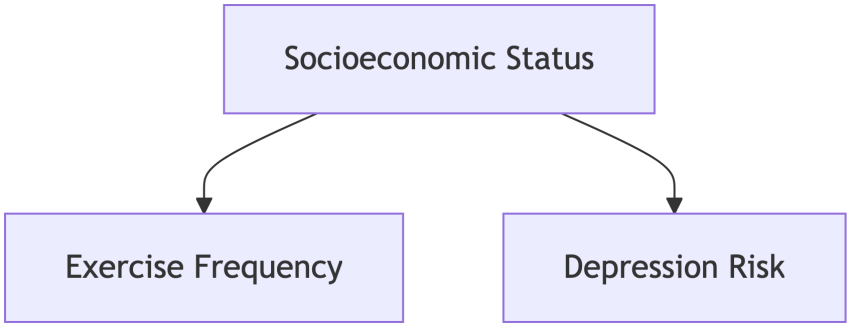
```
graph TD
    E[Education] --> I[Income] --> H[Health]
    E --> HL[Health Literacy] --> H
    E --> SN[Social Networks] --> H
    E --> H
```



Detailed Example 2: Fork (Confounding)

Research Question: Does exercise prevent depression?

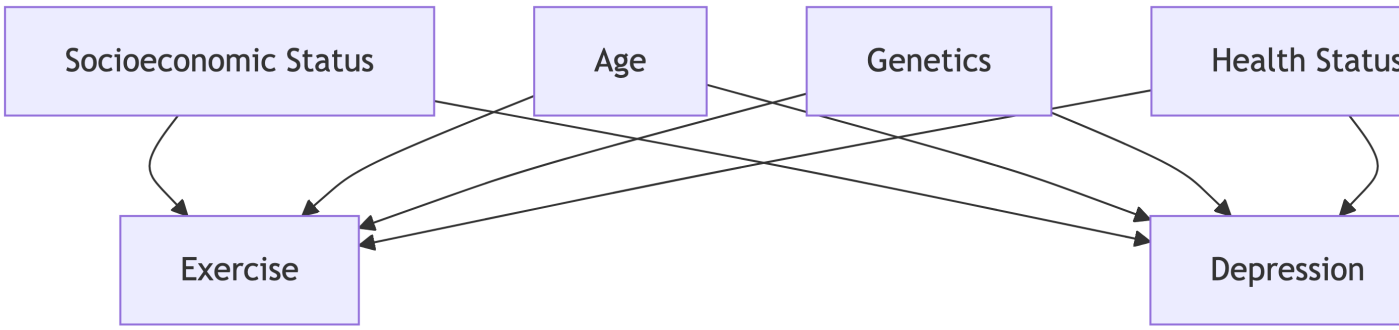
```
graph TD
    SES[Socioeconomic Status] --> E[Exercise Frequency]
    SES --> D[Depression Risk]
```



Confounding Problem: People with higher SES exercise more AND have lower depression risk (better healthcare, less stress, etc.). Without controlling for SES, we might incorrectly conclude that exercise prevents depression when the association is actually due to SES.

Extended Fork with Multiple Confounders:

```
graph TD
    SES[Socioeconomic Status] --> E[Exercise]
    SES --> D[Depression]
    A[Age] --> E
    A --> D
    G[Genetics] --> E
    G --> D
    H[Health Status] --> E
    H --> D
```



R Code: Simulating Chains and Forks

```r
# Simulate Chain Structure: Education → Income → Health
set.seed(123)
n <- 1000

# Chain Simulation
education <- rnorm(n, mean = 12, sd = 3)
income <- 20000 + 3000 * education + rnorm(n, 0, 8000)
health <- 50 + 0.003 * income + rnorm(n, 0, 10)

chain_data <- data.frame(education, income, health)

# Total Effect (don't Control for mediator)
total_model <- lm(health ~ education, data = chain_data)
# Direct Effect (control for mediator)
direct_model <- lm(health ~ education + income, data = chain_data)

cat("=== CHAIN ANALYSIS ===\n")
```

```
=== CHAIN ANALYSIS ===
```

```r
cat(
    "Total effect of education on health:",
    round(coef(total_model)[2], 3),
    "\n"
)
```

```
Total effect of education on health: 9.641
```

```
cat(
    "Direct effect of education on health:",
    round(coef(direct_model)[2], 3),
    "\n"
)
```

```
Direct effect of education on health: -0.175
```

```
cat(
    "Mediated effect through income:",
    round(coef(total_model)[2] - coef(direct_model)[2], 3),
    "\n\n"
)
```

```
Mediated effect through income: 9.816
```

```
# Fork Simulation: SES → Exercise, SES → Depression
ses <- rnorm(n, mean = 0, sd = 1)
exercise <- 2 + 0.8 * ses + rnorm(n, 0, 1) # Higher SES → more exercise
depression <- 5 - 0.6 * ses + rnorm(n, 0, 1.5) # Higher SES → less depression
# Note: Exercise Doesn't Actually Cause Depression in This Simulation

fork_data <- data.frame(ses, exercise, depression)

# Biased Estimate (don't Control for confounder)
biased_model <- lm(depression ~ exercise, data = fork_data)
# Unbiased Estimate (control for confounder)
unbiased_model <- lm(depression ~ exercise + ses, data = fork_data)

cat("=== FORK ANALYSIS ===\n")
```

```
=== FORK ANALYSIS ===
```

```
cat(
    "Biased effect (not controlling for SES):",
```

```
    round(coef(biased_model)[2], 3),
    "\n"
)
```

```
Biased effect (not controlling for SES): -0.245
```

```
cat(
    "Unbiased effect (controlling for SES):",
    round(coef(unbiased_model)[2], 3),
    "\n"
)
```

```
Unbiased effect (controlling for SES): 0.021
```

```
cat(
    "True causal effect should be ~0 since we simulated no causal relationship\n"
)
```

```
True causal effect should be ~0 since we simulated no causal relationship
```

```
# Visualize the Confounding
ggplot(fork_data, aes(x = exercise, y = depression, color = ses)) +
    geom_point(alpha = 0.6) +
    geom_smooth(
        method = "lm",
        se = FALSE,
        aes(group = 1),
        color = "red",
        linetype = "dashed"
    ) +
    geom_smooth(method = "lm", se = FALSE) +
    labs(
        title = "Fork Structure: Confounding by SES",
        subtitle = "Red line = biased association; Blue lines = within SES levels",
        x = "Exercise Frequency",
        y = "Depression Score",
        color = "SES"
    )
```

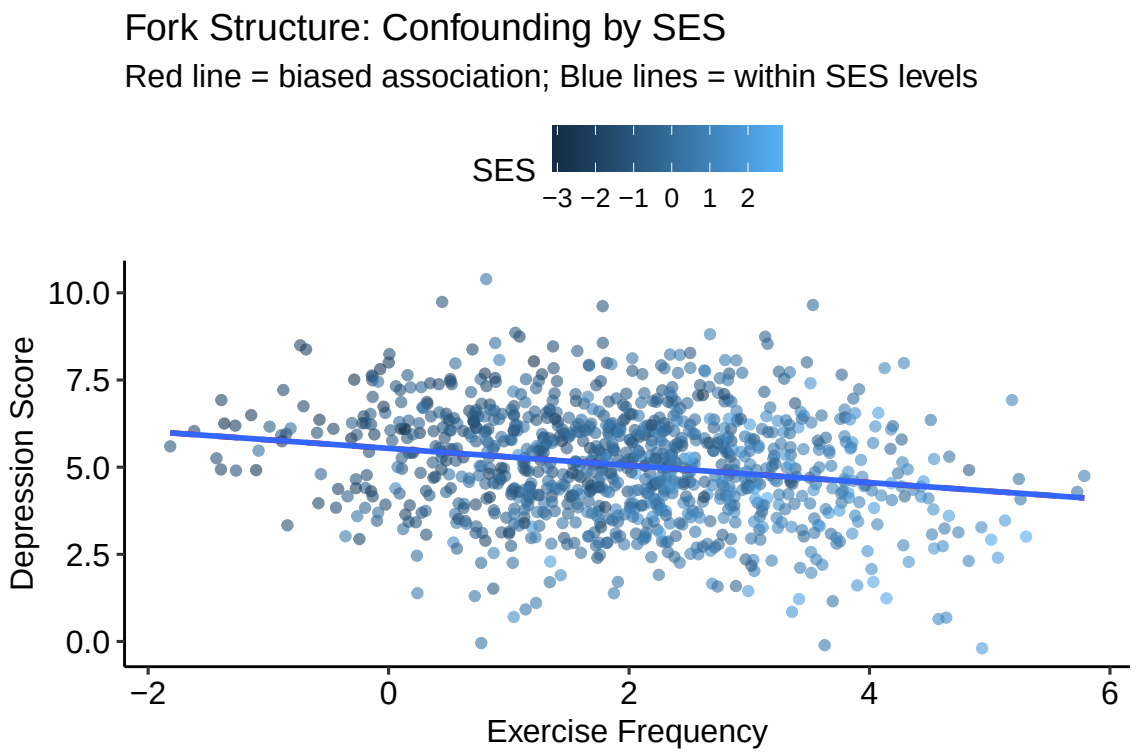


Figure 5: Code for Simulating Chains and Forks

Code Explanation

- In the **chain simulation**, controlling for income (mediator) reduces the education coefficient because we're blocking the indirect pathway
- In the **fork simulation**, the biased model shows a negative association between exercise and depression, but this disappears when we control for SES
- The visualization shows how the overall association (red line) differs from the within-group associations (blue lines)

3.6 Colliders and Their Descendants

Comprehensive Explanation

A **Collider** is a variable that is caused by two or more other variables. In the basic three-node structure, it appears as: $X \rightarrow Z \leftarrow Y$. Colliders have fundamentally different properties from chains and forks and are often the source of the most counterintuitive results in causal inference.

Key Properties of Colliders:

1. **Marginal Independence**: X and Y are **independent** when not conditioning on Z
2. **Conditional Dependence**: X and Y become **dependent** when conditioning on Z
3. **Selection Bias**: Conditioning on colliders creates spurious associations
4. **Blocking Rule**: Collider paths are **blocked** when we don't condition on the collider
5. **Opening Rule**: Collider paths become **open** when we condition on the collider or its descendants

Mathematical Representation: If Z is a collider for X and Y: - $X \perp Y$ (marginally independent) - $X \not\perp Y|Z$ (conditionally dependent given Z)

Types of Collider Bias:

1. **Selection Bias**: Conditioning on a collider that determines sample inclusion
2. **Berkson's Paradox**: Negative correlation among hospital patients between diseases that are positively correlated in the general population
3. **M-bias**: Conditioning on a collider that's also affected by unmeasured confounders
4. **Survivorship Bias**: Conditioning on survival when survival is affected by multiple factors

Descendants of Colliders: Any variable that is causally downstream from a collider inherits its bias-inducing properties. If Z is a collider and W is caused by Z, then conditioning on W also creates spurious associations between the causes of Z.

Formal Rule: In the structure $X \rightarrow Z \leftarrow Y \rightarrow W$, conditioning on W (descendant of collider Z) will induce association between X and Y.

Intuitive Clarification

The "Meeting Point" Analogy: Think of a collider as a **meeting point** where different causal streams converge. Imagine two independent hiking trails (X and Y) that meet at a mountain peak (Z). The trails themselves don't affect each other, but if you only observe hikers at the peak (condition on Z), you might notice patterns—if fewer people came from trail X today, there might be more from trail Y, creating an apparent negative relationship.

The "Selective Restaurant" Analogy: Consider a fancy restaurant (collider) that attracts people who are either very wealthy OR very charming. Among restaurant patrons only, wealth and charm appear negatively correlated—not because wealth reduces charm in the population, but because the restaurant selection creates this artificial relationship.

> ⚠ Major Pitfall: The Collider Trap
>
> Collider bias is often counterintuitive because conditioning (controlling for variables) usually reduces bias, but with colliders, it **creates** bias. Many researchers instinctively want to "control for everything," but controlling for colliders can make your estimates worse, not better.

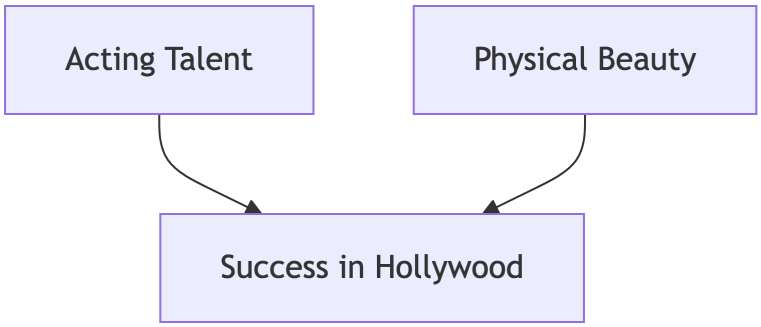
Identifying Colliders in Practice

Ask yourself: "Is this variable caused by both my treatment and other factors?" If yes, it's likely a collider. Common examples: - **Hospital admission** (caused by both disease severity and insurance status) - **Academic success** (caused by both intelligence and effort) - **Employment status** (caused by both skills and economic conditions) - **Marriage status** (caused by both attractiveness and personality)

Detailed Example 1: The Hollywood Talent Paradox

Scenario: Does talent predict success in Hollywood?

```
graph TD
    T[Acting Talent] --> S[Success in Hollywood]
    B[Physical Beauty] --> S
```



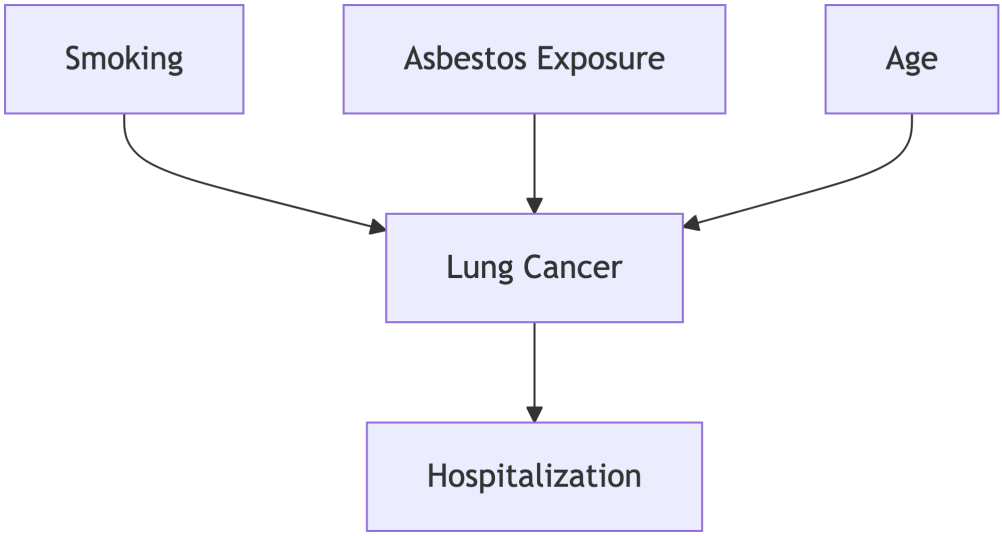
Analysis: - **Population level**: Talent and beauty are unrelated (independent traits) - **Among successful actors**: Talent and beauty appear negatively correlated - **Explanation**: Success requires either high talent OR high beauty (or both). Among successful actors, those with less beauty tend to have more talent to compensate.

Real-world Implications: - Casting directors might mistakenly think talented actors are less attractive - This is pure selection bias—the negative correlation exists only within the selected group

Detailed Example 2: Medical Research Bias

Scenario: Do smoking and asbestos exposure interact to cause lung cancer?

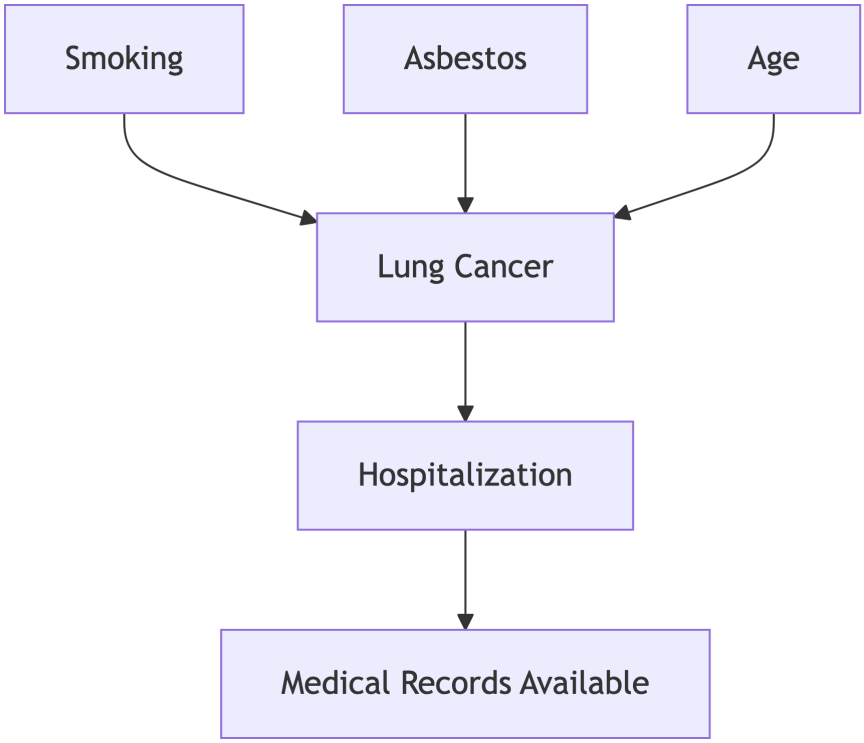
```
graph TD
    S[Smoking] --> LC[Lung Cancer]
    A[Asbestos Exposure] --> LC
    LC --> H[Hospitalization]
    Age[Age] --> LC
```



Hospital-based Study Problem: - If we study only hospitalized patients, we're conditioning on a collider (hospitalization) - Among hospitalized patients, smoking and asbestos exposure might appear negatively associated - This doesn't reflect their true relationship in the population

Extended Collider with Descendant:

```
graph TD
    S[Smoking] --> LC[Lung Cancer]
    A[Asbestos] --> LC
    LC --> H[Hospitalization]
    H --> M[Medical Records Available]
    Age[Age] --> LC
```

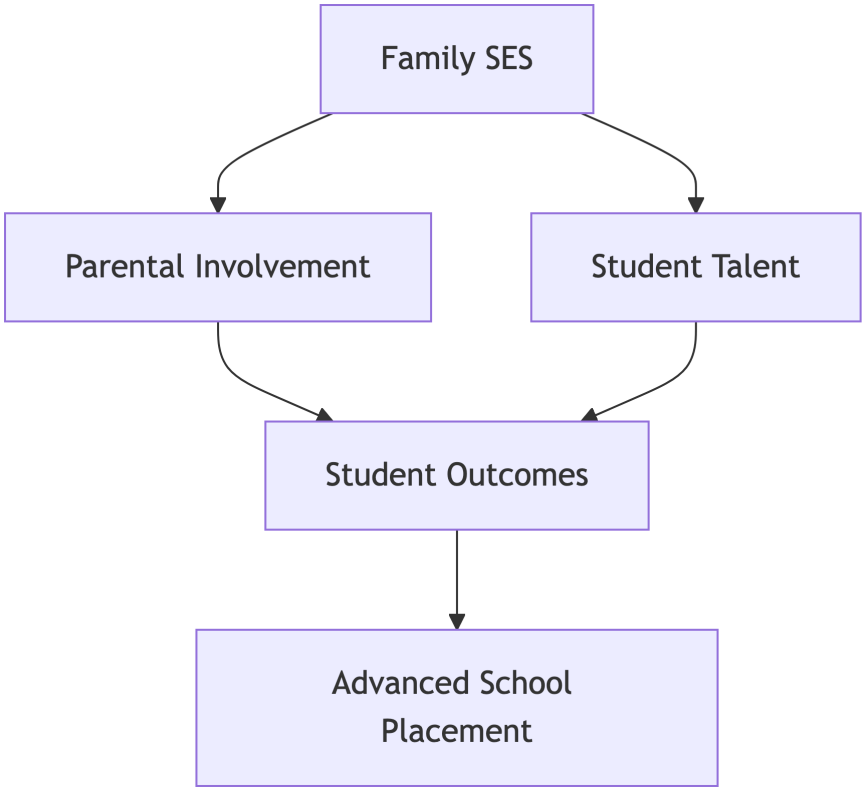


Even conditioning on "medical records available" creates bias because it's a descendant of the collider "hospitalization."

Detailed Example 3: Educational Research Bias

Scenario: Does parental involvement improve student outcomes?

```
graph TD
    PI[Parental Involvement] --> SO[Student Outcomes]
    ST[Student Talent] --> SO
    SO --> AS[Advanced School Placement]
    SES[Family SES] --> PI
    SES --> ST
```



Selection Bias in Advanced Programs: - If we study only students in advanced programs, we're conditioning on a collider - Among advanced students, parental involvement and student talent might appear negatively correlated - High-talent students might succeed despite less parental involvement, while lower-talent students need more parental support to reach advanced programs

R Code: Simulating Collider Bias

```
# Simulate the Hollywood Talent Paradox
set.seed(456)
n <- 10000

# Generate Independent Talent and Beauty
talent <- rnorm(n, mean = 0, sd = 1)
beauty <- rnorm(n, mean = 0, sd = 1)

# Success Depends on EITHER Talent OR Beauty (or both)
```

```r
# This Makes Success a Collider
success_prob <- plogis(-2 + 1.2 * talent + 1.2 * beauty)
success <- rbinom(n, 1, success_prob)

# Create Dataframe
hollywood_data <- data.frame(talent, beauty, success)

# Marginal Correlation (should Be ~0)
marginal_cor <- cor(hollywood_data$talent, hollywood_data$beauty)
cat("Marginal correlation (talent, beauty):", round(marginal_cor, 3), "\n")
```

```
Marginal correlation (talent, beauty): -0.011
```

```r
# Correlation among Successful Actors (should Be negative!)
successful_actors <- hollywood_data[hollywood_data$success == 1, ]
conditional_cor <- cor(successful_actors$talent, successful_actors$beauty)
cat(
    "Conditional correlation among successful actors:",
    round(conditional_cor, 3),
    "\n"
)
```

```
Conditional correlation among successful actors: -0.195
```

```r
# Visualize the Collider Bias
ggplot(hollywood_data, aes(x = talent, y = beauty, color = factor(success))) +
    geom_point(alpha = 0.6) +
    geom_smooth(method = "lm", se = FALSE) +
    facet_wrap(~ factor(success, labels = c("All Actors", "Successful Only"))) +
    labs(
        title = "Collider Bias: Hollywood Talent Paradox",
        subtitle = "Talent and beauty appear negatively correlated among successful
        actors",
        x = "Acting Talent",
        y = "Physical Beauty",
        color = "Success"
    ) +
    theme(legend.position = "none")
```

```r
# Demonstrate Descendant Bias
# Add "fame" as Descendant of Success
fame <- success * rnorm(n, mean = 5, sd = 2) + rnorm(n, 0, 1)
hollywood_data$fame <- pmax(fame, 0) # Fame can't be negative

# Even Conditioning on Fame (descendant) Creates Bias
high_fame <- hollywood_data[
    hollywood_data$fame > quantile(hollywood_data$fame, 0.8),
]
descendant_cor <- cor(high_fame$talent, high_fame$beauty)
cat(
    "Correlation among high-fame actors (descendant bias):",
    round(descendant_cor, 3),
    "\n"
)
```

```
Correlation among high-fame actors (descendant bias): -0.135
```

```r
# Medical Research Example: Hospital Selection Bias
# Simulate Smoking, Asbestos, and Lung Cancer
smoking <- rbinom(n, 1, 0.3)
asbestos <- rbinom(n, 1, 0.1)
age <- rnorm(n, 65, 10)

# Lung Cancer Depends on All Three (but Smoking and Asbestos Are independent)
lc_prob <- plogis(-4 + 2 * smoking + 1.5 * asbestos + 0.05 * (age - 65))
lung_cancer <- rbinom(n, 1, lc_prob)

# Hospitalization Depends on Lung Cancer (collider)
hosp_prob <- plogis(-2 + 3 * lung_cancer + 0.02 * (age - 65))
hospitalized <- rbinom(n, 1, hosp_prob)

medical_data <- data.frame(smoking, asbestos, lung_cancer, hospitalized, age)

# Population-level Association (should Be ~0)
pop_table <- table(medical_data$smoking, medical_data$asbestos)
pop_chi2 <- chisq.test(pop_table)
```

```r
cat(
    "\nPopulation: Smoking-Asbestos association p-value:",
    round(pop_chi2$p.value, 3),
    "\n"
)
```

```
Population: Smoking-Asbestos association p-value: 0.797
```

```r
# Hospital-based Study (conditioning on collider)
hospital_data <- medical_data[medical_data$hospitalized == 1, ]
hosp_table <- table(hospital_data$smoking, hospital_data$asbestos)
hosp_chi2 <- chisq.test(hosp_table)
cat(
    "Hospital sample: Smoking-Asbestos association p-value:",
    round(hosp_chi2$p.value, 3),
    "\n"
)
```

```
Hospital sample: Smoking-Asbestos association p-value: 0.007
```

```r
# Visualize Selection Bias
medical_data$sample <- ifelse(
    medical_data$hospitalized == 1,
    "Hospital Sample",
    "Population"
)
ggplot(medical_data, aes(x = factor(smoking), fill = factor(asbestos))) +
    geom_bar(position = "fill") +
    facet_wrap(~sample) +
    labs(
        title = "Selection Bias in Hospital-based Studies",
        subtitle = "Smoking and asbestos appear associated in hospital sample",
        x = "Smoking Status",
        y = "Proportion",
        fill = "Asbestos\nExposure"
    )
```

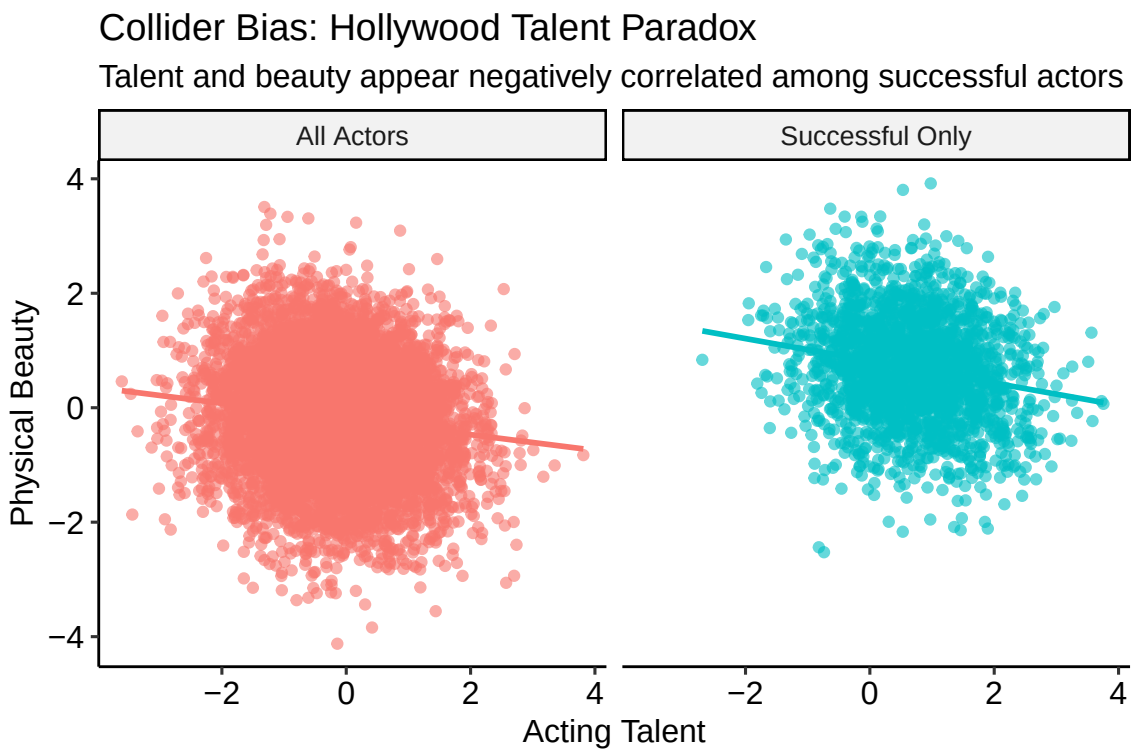

Collider Bias: Hollywood Talent Paradox
Talent and beauty appear negatively correlated among successful actors
All Actors
Successful Only
Physical Beauty
Acting Talent
4
2
0
−2
−4
−2
0
2
4

Figure 6

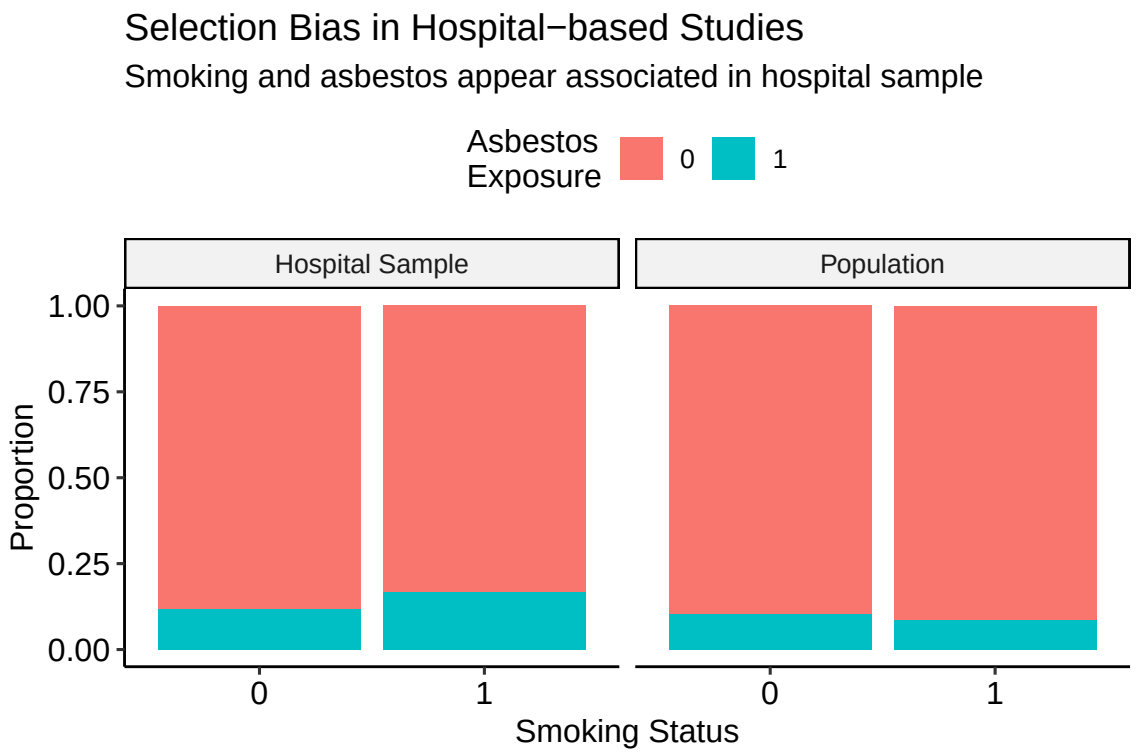


Figure 7

i Code Interpretation

- **Hollywood example**: Talent and beauty are uncorrelated in the population but negatively correlated among successful actors
- **Medical example**: Independent risk factors appear associated when we condition on the outcome (selection bias)
- **Descendant effect**: Even conditioning on variables caused by colliders creates bias
- These simulations demonstrate why "controlling for everything" can be harmful

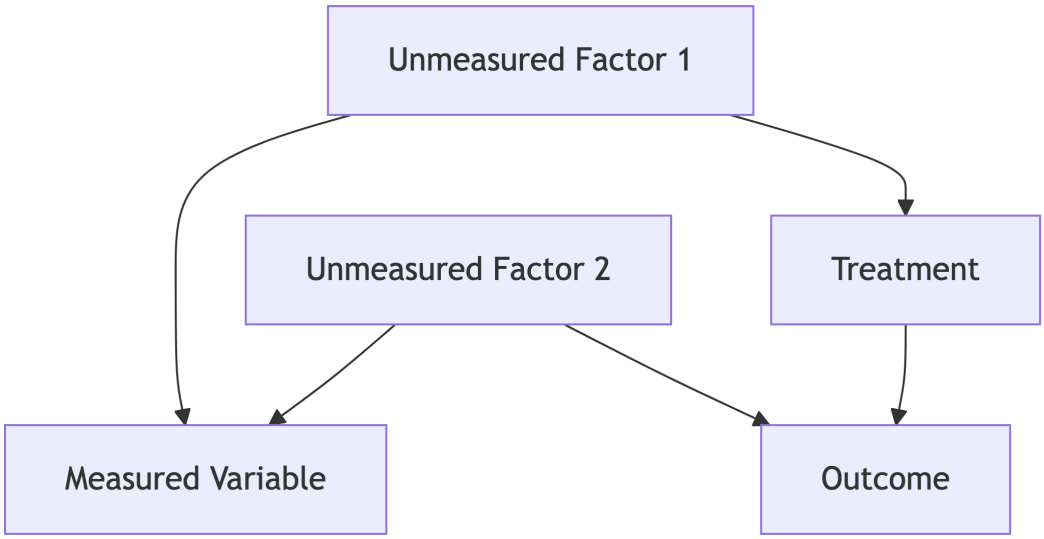
Key Assumptions About Colliders

! When Colliders Become Confounders

Sometimes a variable can be both a collider AND a confounder. This creates a dilemma: controlling for it reduces confounding bias but increases collider bias. Advanced methods like instrumental variables or sensitivity analysis may be needed.

Example: M-bias Structure

```
graph TD
    U1[Unmeasured Factor 1] --> M[Measured Variable]
    U2[Unmeasured Factor 2] --> M
    U1 --> T[Treatment]
    U2 --> O[Outcome]
    T --> O
```



In this case, M is a collider for $U1$ and $U2$, but controlling for M opens a backdoor path $T \leftarrow U1 \rightarrow M \leftarrow U2 \rightarrow O$.

3.7 D-separation

Comprehensive Explanation

d-separation (directional separation) is a fundamental graphical criterion that determines when two sets of variables are conditionally independent given a third set, based purely on the structure of the DAG. It's the key tool for reading conditional independence relationships directly from causal graphs.

Formal Definition: Two disjoint sets of nodes X and Y are **d-separated** by a set Z if every path between every node in X and every node in Y is **blocked** by Z.

The Three Blocking Rules: A path between X and Y is blocked by conditioning set Z if the path contains at least one of these configurations:

1. **Chain/Mediator**: $A \to B \to C$ where B $\in$ Z
   - Conditioning on the middle node blocks the flow
   - "Stopping at the mediator"
2. **Fork/Confounder**: $A \leftarrow B \to C$ where B $\in$ Z
   - Conditioning on the common cause blocks the spurious association
   - "Controlling for the confounder"
3. **Collider**: $A \to B \leftarrow C$ where B $\notin$ Z AND no descendant of B is in Z
   - The collider naturally blocks the flow when not conditioned upon
   - "Avoiding selection bias"

The d-separation Theorem: If sets X and Y are d-separated by set Z in a DAG G, then X $\perp$ Y | Z in every probability distribution P that is Markov and faithful to G.

Practical Importance: - d-separation allows us to read conditional independence statements directly from the graph - It helps identify which variables to control for in regression analysis - It's the foundation for identifying valid adjustment sets

Intuitive Clarification

Think of d-separation as **"information flow"** through a network of pipes (the DAG). Information (statistical association) flows along paths unless those paths are blocked by valves (conditioning).

Different structures behave like different types of valves: - **Chains/Forks**: Normal valves - turn them "on" (condition) to stop the flow - **Colliders**: Reverse valves - they're "off" by default, turn them "on" (condition) to start the flow

The "d" in d-separation: Originally stood for "directional" but can be remembered as: - **D**etermining conditional independence - **D**irected paths and their blocking - **D**eciding what to control for

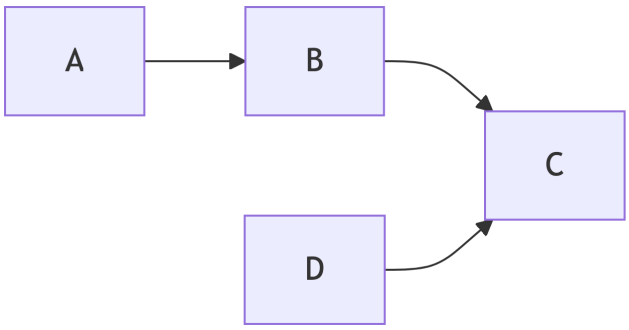
Practical D-separation Algorithm

1. **List all paths** from X to Y (ignoring arrow directions)
2. **Check each path** for blocking according to the three rules
3. **If ALL paths are blocked** by Z, then X and Y are d-separated by Z
4. **If ANY path remains open**, then X and Y are NOT d-separated by Z

Detailed Example 1: Simple D-separation

Consider the DAG: $A \to B \to C \leftarrow D$

```
graph LR
    A --> B --> C
    D --> C
```



Question 1: Are A and D d-separated by $\emptyset$ (empty set)? - **Path**: A → B → C ← D - **Analysis**: Contains collider at C, and C is not in conditioning set - **Result**: Path is blocked → A $\perp$ D (marginally independent)

Question 2: Are A and D d-separated by {B}? - **Path**: A → B → C ← D
- **Analysis**: Contains collider at C, C $\notin$ {B}, no descendant of C in {B} - **Result**: Path is blocked → A $\perp$ D | B

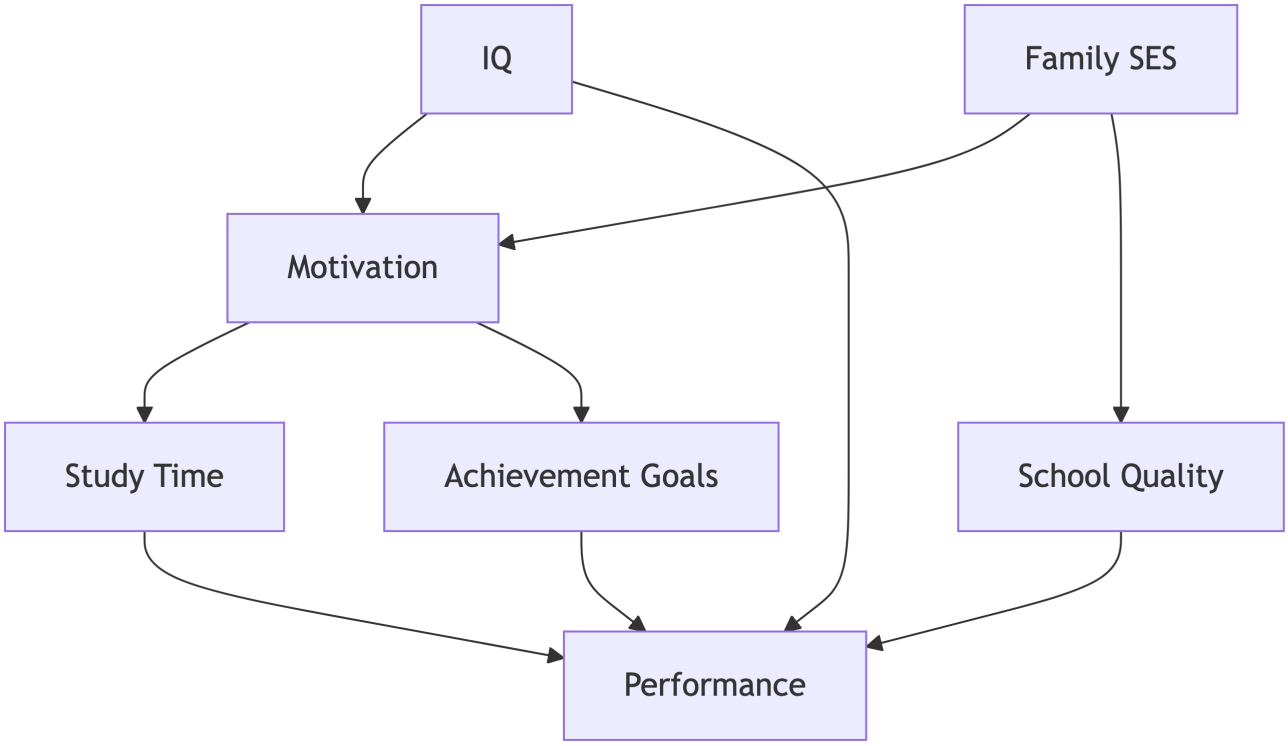
Question 3: Are A and D d-separated by {C}? - **Path**: A → B → C ← D - **Analysis**: Contains collider at C, but now C $\in$ {C} - **Result**: Path is NOT blocked → A $\not\perp$ D | C (conditioning on collider opens path)

Detailed Example 2: Complex Network

Consider this more complex DAG representing factors affecting student performance:

```
graph TD
    SES[Family SES] --> M[Motivation]
    SES --> S[School Quality]
    IQ --> M
    IQ --> P[Performance]
    M --> ST[Study Time]
```

```
    ST --> P
    S --> P
    M --> A[Achievement Goals]
    A --> P
```



Question: Are IQ and School Quality d-separated by {Family SES}?

Paths from IQ to School Quality: 1. IQ → Motivation ← Family SES → School Quality 2. IQ → Performance ← School Quality (this path goes through Performance) 3. IQ → Motivation → Study Time → Performance ← School Quality 4. IQ → Motivation → Achievement Goals → Performance ← School Quality

Analysis: - **Path 1**: Contains collider at Motivation, Motivation $\notin$ {SES} → BLOCKED - **Paths 2-4**: All go through Performance, which is a collider for paths coming from IQ and School Quality → BLOCKED

Result: IQ $\perp$ School Quality | Family SES

R Code: D-separation Analysis

```
# Create the Complex Student Performance DAG
student_dag <- dagitty(
    "dag {
  FamilySES → Motivation → StudyTime → Performance
  FamilySES → SchoolQuality → Performance
  IQ → Motivation → AchievementGoals → Performance
  IQ → Performance
}"
)

# Set Coordinates for Better Visualization
coordinates(student_dag) <- list(
    x = c(
        FamilySES = 0,
        IQ = 0,
        Motivation = 1,
        SchoolQuality = 1,
        StudyTime = 2,
        AchievementGoals = 2,
        Performance = 3
    ),
    y = c(
        FamilySES = 0,
        IQ = 2,
        Motivation = 1,
        SchoolQuality = 0,
        StudyTime = 1.5,
        AchievementGoals = 0.5,
        Performance = 1
    )
)

# Visualize the DAG
ggdag(student_dag) +
    theme_dag_blank() +
    labs(title = "Student Performance Causal DAG")
```

```
# Test Various D-separation Queries
cat("=== D-SEPARATION ANALYSIS ===\n")
```

```
=== D-SEPARATION ANALYSIS ===
```

```
# Are IQ and SchoolQuality D-separated by FamilySES?
ds1 <- dseparated(student_dag, "IQ", "SchoolQuality", "FamilySES")
cat("IQ ⫫ SchoolQuality | FamilySES:", ds1, "\n")
```

```
IQ ⫫ SchoolQuality | FamilySES: TRUE
```

```
# Are IQ and SchoolQuality D-separated by Nothing?
ds2 <- dseparated(student_dag, "IQ", "SchoolQuality", character(0))
cat("IQ ⫫ SchoolQuality (marginal):", ds2, "\n")
```

```
IQ ⫫ SchoolQuality (marginal): TRUE
```

```
# Are IQ and Performance D-separated by StudyTime?
ds3 <- dseparated(student_dag, "IQ", "Performance", "StudyTime")
cat("IQ ⫫ Performance | StudyTime:", ds3, "\n")
```

```
IQ ⫫ Performance | StudyTime: FALSE
```

```
# Are IQ and Performance D-separated by Motivation?
ds4 <- dseparated(student_dag, "IQ", "Performance", "Motivation")
cat("IQ ⫫ Performance | Motivation:", ds4, "\n")
```

```
IQ ⫫ Performance | Motivation: FALSE
```

```
# Find All Implied Conditional Independencies
all_implications <- impliedConditionalIndependencies(student_dag)
cat("\nAll implied conditional independencies:\n")
```

```

All implied conditional independencies:
```

```
for (i in 1:length(all_implications)) {
    cat(i, ":", toString(all_implications[[i]]), "\n")
}
```

```
1 : AchG _||_ FSES | Mtvt
2 : AchG _||_ IQ | Mtvt
3 : AchG _||_ SchQ | FSES
4 : AchG _||_ SchQ | Mtvt
5 : AchG _||_ StdT | Mtvt
6 : FSES _||_ IQ
7 : FSES _||_ Prfr | AchG, IQ, SchQ, StdT
8 : FSES _||_ Prfr | IQ, Mtvt, SchQ
9 : FSES _||_ StdT | Mtvt
10 : IQ _||_ SchQ
11 : IQ _||_ StdT | Mtvt
12 : Mtvt _||_ Prfr | AchG, IQ, SchQ, StdT
13 : Mtvt _||_ Prfr | AchG, FSES, IQ, StdT
14 : Mtvt _||_ SchQ | FSES
15 : SchQ _||_ StdT | Mtvt
16 : SchQ _||_ StdT | FSES
```

```
# Test a Collider Example
collider_dag <- dagitty("dag { X → Z <- Y }")

cat("\n═══ COLLIDER EXAMPLE ═══\n")
```

```
═══ COLLIDER EXAMPLE ═══
```

```
cat("X ⫫ Y (marginal):", dseparated(collider_dag, "X", "Y", character(0)), "\n")
```

```
X ⫫ Y (marginal): TRUE
```

```
cat("X ⫫ Y | Z:", dseparated(collider_dag, "X", "Y", "Z"), "\n")
```

```
X ⫫ Y | Z: FALSE
```

```
# Example with Descendant of Collider
desc_dag <- dagitty("dag { X → Z <- Y; Z → W }")
cat("X ⫫ Y | W (descendant):", dseparated(desc_dag, "X", "Y", "W"), "\n")
```

```
X ⫫ Y | W (descendant): FALSE
```

```
# More Complex Example with Multiple Paths
complex_dag <- dagitty(
    "dag {
  A → B → D
  A → C → D
  B → E <- C
  E → F
}"
)

# Visualize
ggdag(complex_dag) + theme_dag_blank() + labs(title = "Complex Multi-path DAG")

cat("\n═══ COMPLEX PATH ANALYSIS ═══\n")
```

```
═══ COMPLEX PATH ANALYSIS ═══
```

```
# Are A and D D-separated by B?
cat("A ⫫ D | B:", dseparated(complex_dag, "A", "D", "B"), "\n")
```

```
A ⫫ D | B: FALSE
```

```
# Are A and D D-separated by E?
cat("A ⫫ D | E:", dseparated(complex_dag, "A", "D", "E"), "\n")
```

```
A ⫫ D | E: FALSE
```

```
# Are A and D D-separated by {B,C}?
cat("A ⫫ D | {B,C}:", dseparated(complex_dag, "A", "D", c("B", "C")), "\n")
```

A ⫫ D | {B,C}: TRUE

```
# Find Paths between A and D
paths_AD <- paths(complex_dag, "A", "D")
cat("\nPaths from A to D:\n")
```

```
Paths from A to D:
```

```
print(paths_AD)
```

```
$paths
[1] "A -> B -> D"           "A -> B -> E <- C -> D" "A -> C -> D"
[4] "A -> C -> E <- B -> D"

$open
[1]  TRUE FALSE  TRUE FALSE
```

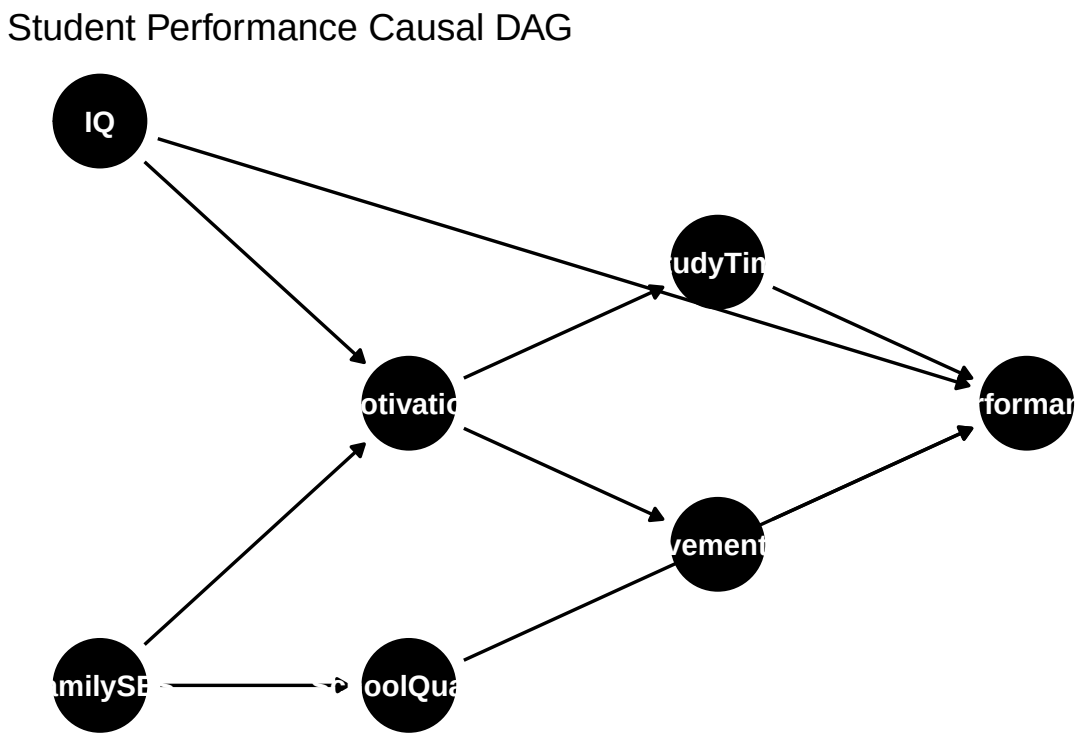


Figure 8: Code for D-separation Analysis

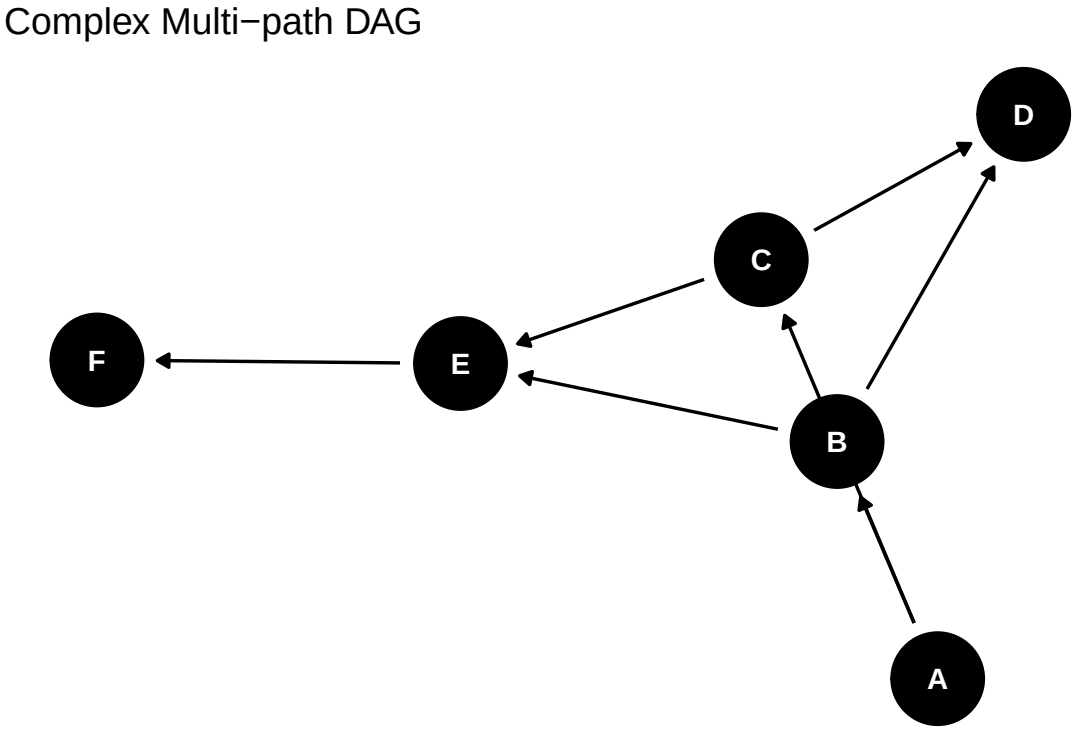


Figure 9: Code for D-separation Analysis

Code Explanation

- `dseparated()` tests whether two variables are d-separated given a conditioning set
- `impliedConditionalIndependencies()` lists all conditional independence relationships implied by the DAG
- `paths()` shows all paths between two variables
- Notice how colliders behave opposite to chains/forks in terms of conditioning

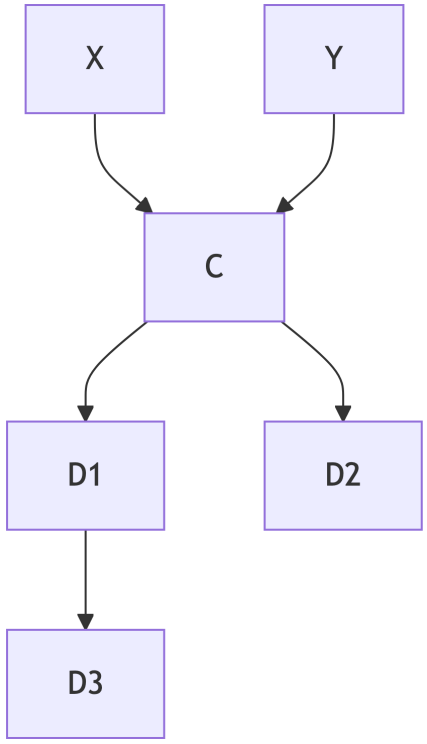
Advanced D-separation Rules

Descendant Rule

If Z is a collider on a path, then the path is blocked if and only if: 1. Z is NOT in the conditioning set, AND
2. NO descendant of Z is in the conditioning set
This means conditioning on any descendant of a collider opens the collider path.

Example with Descendants:

```
graph TD
    X --> C
    Y --> C
    C --> D1
    C --> D2
    D1 --> D3
```



- Conditioning on C opens the X-Y path (collider bias)
- Conditioning on D1, D2, or D3 also opens the X-Y path (descendant bias)
- To keep X and Y independent, don't condition on C or any of its descendants

3.8 Flow of Association and Causation

Comprehensive Explanation

Understanding how **association** and **causation** flow through DAGs is the culmination of causal graph theory. This knowledge enables us to identify valid strategies for causal inference and avoid the many pitfalls that can bias our estimates.

Fundamental Principles:

1. **Causal Flow**: Causal effects flow only along directed paths in the direction of the arrows
2. **Associational Flow**: Statistical associations can flow along any path connecting two variables, regardless of arrow directions
3. **Blocking and Opening**: Conditioning on variables can block or open these flows according to d-separation rules
4. **Confounding**: Spurious associations arise from non-causal paths between treatment and outcome

Types of Paths in Causal Analysis:

1. **Causal Paths (Front-door paths)**:
   - Start with an arrow OUT of the treatment
   - Follow directed edges toward the outcome
   - Represent genuine causal mechanisms
   - Should be kept OPEN to preserve causal effects
2. **Confounding Paths (Back-door paths)**:
   - Start with an arrow INTO the treatment
   - Connect treatment and outcome through common causes
   - Create spurious associations
   - Should be BLOCKED by conditioning
3. **Selection Bias Paths**:
   - Pass through colliders
   - Are naturally blocked (good!)
   - Become opened when we condition on colliders (bad!)

The Golden Rules for Causal Inference: 1. **Block all back-door paths** (eliminate confounding) 2. **Keep all causal paths open** (preserve the effect) 3. **Don't open collider paths** (avoid selection bias)

Intuitive Clarification

Think of your DAG as a **city traffic system**: - **Causal effects** are like delivery trucks that must follow one-way streets (arrows) - **Statistical association** is like information/rumors that can travel in any direction through connected neighborhoods - **Conditioning** is like setting up police checkpoints that stop certain types of traffic - Your goal is to block all the "backdoor routes" (confounding) while keeping the "main route" (causation) clear

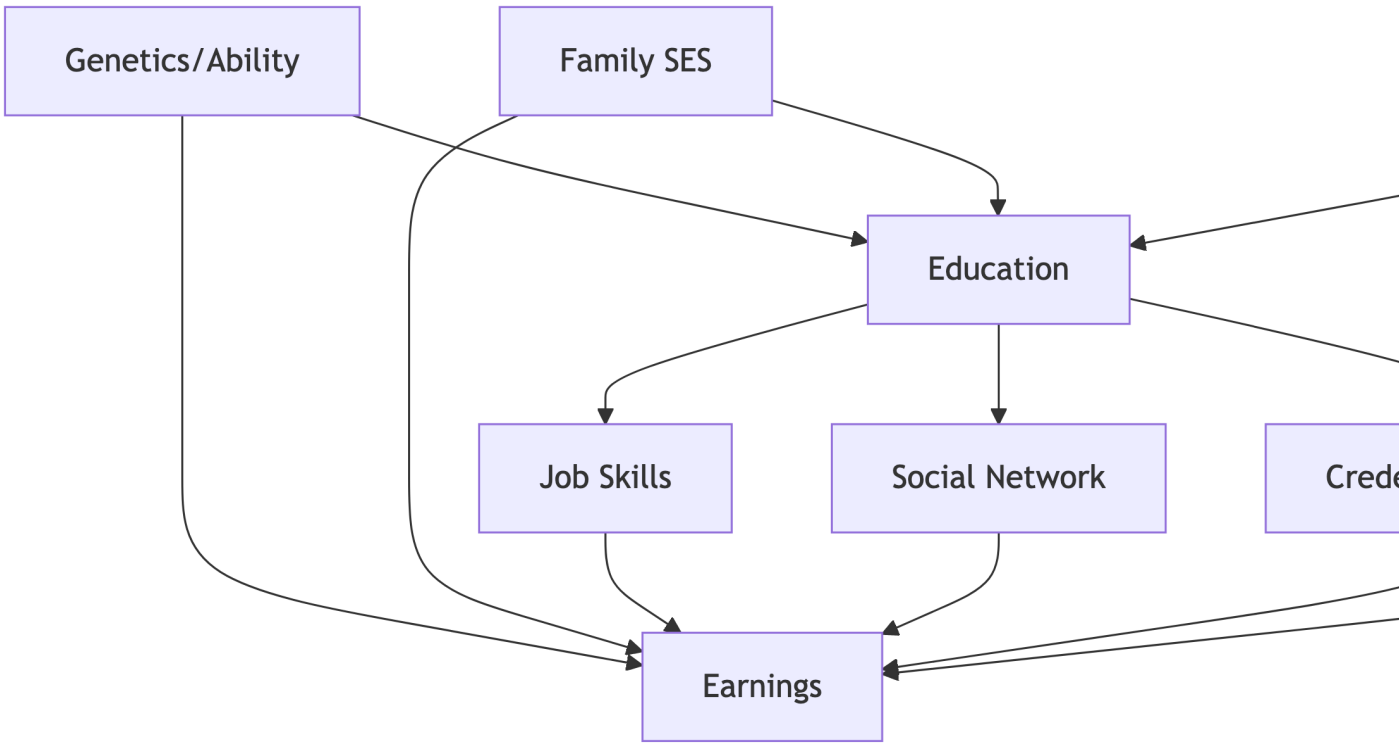
! The Fundamental Problem

We observe **statistical associations** in data, but we want to infer **causal relationships**. DAGs help us understand which associations reflect causation and which reflect confounding, allowing us to design analysis strategies that isolate causal effects.

Detailed Example 1: Education and Earnings

Research Question: What is the causal effect of education on lifetime earnings?

```
graph TD
    G["Genetics/Ability"] --> E["Education"]
    G --> Earn["Earnings"]
    SES["Family SES"] --> E
    SES --> Earn
    E --> Skills["Job Skills"]
    Skills --> Earn
    E --> Network["Social Network"]
    Network --> Earn
    E --> Signal["Credential Signaling"]
    Signal --> Earn
    Loc[Location] --> E
    Loc --> Earn
```



Path Analysis:

Causal Paths (Front-door): 1. Education → Job Skills → Earnings (human capital) 2. Education → Social Network → Earnings (social capital)
3. Education → Credential Signaling → Earnings (signaling)

Confounding Paths (Back-door): 1. Education ← Genetics → Earnings (ability bias) 2. Education ← Family SES → Earnings (socioeconomic confounding) 3. Education ← Location → Earnings (geographic confounding)

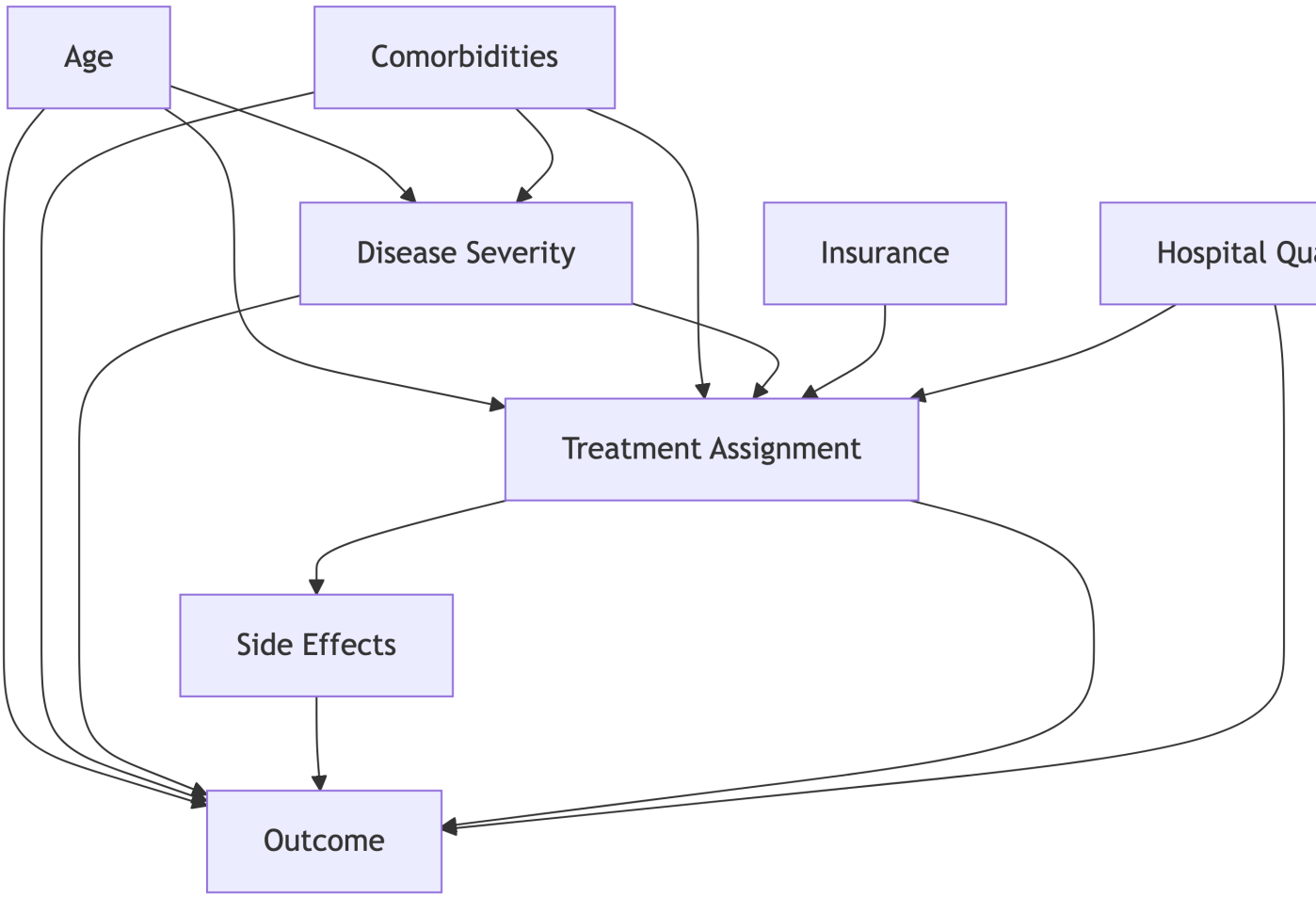
Valid Adjustment Strategy: Control for {Genetics, Family SES, Location} to block all back-door paths while keeping causal paths open.

Invalid Strategies: - Don't control for Job Skills, Social Network, or Signaling (these are mediators on causal paths) - Be careful about controlling for variables that might be colliders

Detailed Example 2: Medical Treatment Effectiveness

Research Question: Does a new drug treatment improve patient outcomes?

```
graph TD
    Age --> S[Disease Severity]
    Age --> T[Treatment Assignment]
    Age --> O[Outcome]
    Comorbid[Comorbidities] --> S
    Comorbid --> T
    Comorbid --> O
    S --> T
    S --> O
    T --> SE[Side Effects]
    SE --> O
    T --> O
    Insurance[Insurance] --> T
    Hosp[Hospital Quality] --> T
    Hosp --> O
```



Confounding Analysis: - Age, Comorbidities: Affect both treatment and outcome (confounders) - Disease Severity: Affects both treatment assignment and outcome (confounder)
- Insurance, Hospital Quality: May affect treatment access and outcomes

Selection Bias Concerns: - If we condition on Side Effects, we might create collider bias - Hospital admission itself might be a collider

Valid Adjustment Strategy: Control for Age, Comorbidities, Disease Severity, Insurance, Hospital Quality

R Code: Comprehensive Flow Analysis

```
# Create the Education-earnings DAG
```

```r
education_dag <- dagitty("dag {
  Genetics → Education → JobSkills → Earnings
  Genetics → Earnings
  FamilySES → Education → SocialNetwork → Earnings
  FamilySES → Earnings
  Education → CredentialSignaling → Earnings
  Location → Education
  Location → Earnings
}")

# Set Layout for Better Visualization
coordinates(education_dag) <- list(
  x = c(Genetics = 0, FamilySES = 0, Location = 0, Education = 2,
        JobSkills = 4, SocialNetwork = 4, CredentialSignaling = 4, Earnings = 6),
  y = c(Genetics = 2, FamilySES = 1, Location = 0, Education = 1,
        JobSkills = 2, SocialNetwork = 1, CredentialSignaling = 0, Earnings = 1)
)

# Visualize with Different Path Types
ggdag_paths(education_dag, "Education", "Earnings") +
  theme_dag_blank() +
  labs(title = "All Paths from Education to Earnings",
       subtitle = "Red = Causal paths, Blue = Confounding paths")

# Find All Paths
all_paths <- paths(education_dag, "Education", "Earnings")
cat("═══ ALL PATHS FROM EDUCATION TO EARNINGS ═══\n")
```

```
═══ ALL PATHS FROM EDUCATION TO EARNINGS ═══
```

```r
print(all_paths)
```

```
$paths
[1] "Education -> CredentialSignaling -> Earnings"
[2] "Education -> JobSkills -> Earnings"
[3] "Education -> SocialNetwork -> Earnings"
[4] "Education <- FamilySES -> Earnings"
[5] "Education <- Genetics -> Earnings"
```

```
[6] "Education <- Location -> Earnings"

$open
[1] TRUE TRUE TRUE TRUE TRUE TRUE
```

```
# Identify Back-door Paths
backdoor_paths <- paths(education_dag, "Education", "Earnings",
                        directed = FALSE)$paths
cat("\n=== BACK-DOOR PATH ANALYSIS ===\n")
```

```

=== BACK-DOOR PATH ANALYSIS ===
```

```
print(backdoor_paths)
```

```
[1] "Education -> CredentialSignaling -> Earnings"
[2] "Education -> JobSkills -> Earnings"
[3] "Education -> SocialNetwork -> Earnings"
[4] "Education <- FamilySES -> Earnings"
[5] "Education <- Genetics -> Earnings"
[6] "Education <- Location -> Earnings"
```

```
# Find Valid Adjustment Sets (back-door criterion)
valid_adjustments <- adjustmentSets(education_dag,
                                    exposure = "Education",
                                    outcome = "Earnings")
cat("\n=== VALID ADJUSTMENT SETS ===\n")
```

```

=== VALID ADJUSTMENT SETS ===
```

```
print(valid_adjustments)
```

```
{ FamilySES, Genetics, Location }
```

```
# Test if Specific Sets Are Valid
test_set1 <- c("Genetics", "FamilySES", "Location")
is_valid1 <- isAdjustmentSet(education_dag, "Education", "Earnings", test_set1)
cat("Is {Genetics, FamilySES, Location} valid?", is_valid1, "\n")
```

```
Is {Genetics, FamilySES, Location} valid? FALSE
```

```
# What Happens if We Mistakenly Control for a Mediator?
test_set2 <- c("Genetics", "FamilySES", "JobSkills")
is_valid2 <- isAdjustmentSet(education_dag, "Education", "Earnings", test_set2)
cat("Is {Genetics, FamilySES, JobSkills} valid?", is_valid2, "\n")
```

```
Is {Genetics, FamilySES, JobSkills} valid? FALSE
```

```
# Visualize Adjustment Sets
ggdag_adjustment_set(education_dag, "Education", "Earnings") +
  theme_dag_blank() +
  labs(title = "Valid Adjustment Sets for Education → Earnings")

# Medical Treatment Example with Collider
medical_dag <- dagitty("dag {
  Age → DiseaseSeverity → Treatment → Outcome
  Age → Treatment → SideEffects <- DiseaseSeverity
  Age → Outcome
  Comorbidities → Treatment
  Comorbidities → Outcome
  DiseaseSeverity → Outcome
}")

cat("\n=== MEDICAL TREATMENT ANALYSIS ===\n")
```

```
=== MEDICAL TREATMENT ANALYSIS ===
```

```
# Find Adjustment Sets for Treatment Effect
medical_adjustments <- adjustmentSets(medical_dag, "Treatment", "Outcome")
print(medical_adjustments)
```

```
{ Age, Comorbidities, DiseaseSeverity }
```

```
# What if We Mistakenly Control for Side Effects (collider)?
controls_se <- isAdjustmentSet(medical_dag, "Treatment", "Outcome",
                               c("Age", "Comorbidities", "DiseaseSeverity",
                               "SideEffects"))
cat("Controlling for SideEffects (collider):", controls_se, "\n")
```

```
Controlling for SideEffects (collider): FALSE
```

```
# Visualize the Medical DAG with Problematic Collider
ggdag(medical_dag) +
  theme_dag_blank() +
  labs(title = "Medical Treatment DAG",
       subtitle = "SideEffects is a collider - don't control for it!")

# Simulate the Bias from Controlling for a Collider
set.seed(789)
n <- 1000

# Simulate Medical Data
age <- rnorm(n, 65, 10)
comorbidities <- rbinom(n, 3, 0.3)
disease_severity <- 1 + 0.02 * age + 0.3 * comorbidities + rnorm(n, 0, 0.5)

# Treatment Assignment Depends on Age, Comorbidities, and Disease Severity
treatment_prob <- plogis(-2 + 0.02 * age + 0.2 * comorbidities + 0.5 *
disease_severity)
treatment <- rbinom(n, 1, treatment_prob)

# Side Effects Depend on both Treatment and Disease Severity (collider!)
se_prob <- plogis(-3 + 2 * treatment + 0.8 * disease_severity)
side_effects <- rbinom(n, 1, se_prob)

# Outcome Depends on Age, Comorbidities, Disease Severity, and Treatment
outcome <- 10 - 0.05 * age - 0.3 * comorbidities - 0.5 * disease_severity +
           1.5 * treatment + rnorm(n, 0, 1)

medical_data <- data.frame(age, comorbidities, disease_severity,
```

```
                          treatment, side_effects, outcome)

# Correct Analysis (don't Control for collider)
correct_model <- lm(outcome ~ treatment + age + comorbidities + disease_severity,
                    data = medical_data)

# Biased Analysis (controlling for collider)
biased_model <- lm(outcome ~ treatment + age + comorbidities + disease_severity +
side_effects,
                   data = medical_data)

cat("\n═══ COLLIDER BIAS DEMONSTRATION ═══\n")
```

```
═══ COLLIDER BIAS DEMONSTRATION ═══
```

```
cat("True treatment effect (correct model):",
round(coef(correct_model)["treatment"], 3), "\n")
```

```
True treatment effect (correct model): 1.363
```

```
cat("Biased treatment effect (controlling for collider):",
round(coef(biased_model)["treatment"], 3), "\n")
```

```
Biased treatment effect (controlling for collider): 1.403
```

```
cat("Bias introduced by controlling for collider:",
    round(coef(biased_model)["treatment"] - coef(correct_model)["treatment"], 3),
    "\n")
```

```
Bias introduced by controlling for collider: 0.039
```

```
# Demonstrate why the Bias Occurs
# Among Patients with Side Effects, Treatment and Disease Severity Are Negatively
Associated
se_patients <- medical_data[medical_data$side_effects == 1, ]
```

```
cat("Correlation between treatment and disease severity among patients with side
effects:",
    round(cor(se_patients$treatment, se_patients$disease_severity), 3), "\n")
```

```
Correlation between treatment and disease severity among patients with side effects: 0.051
```

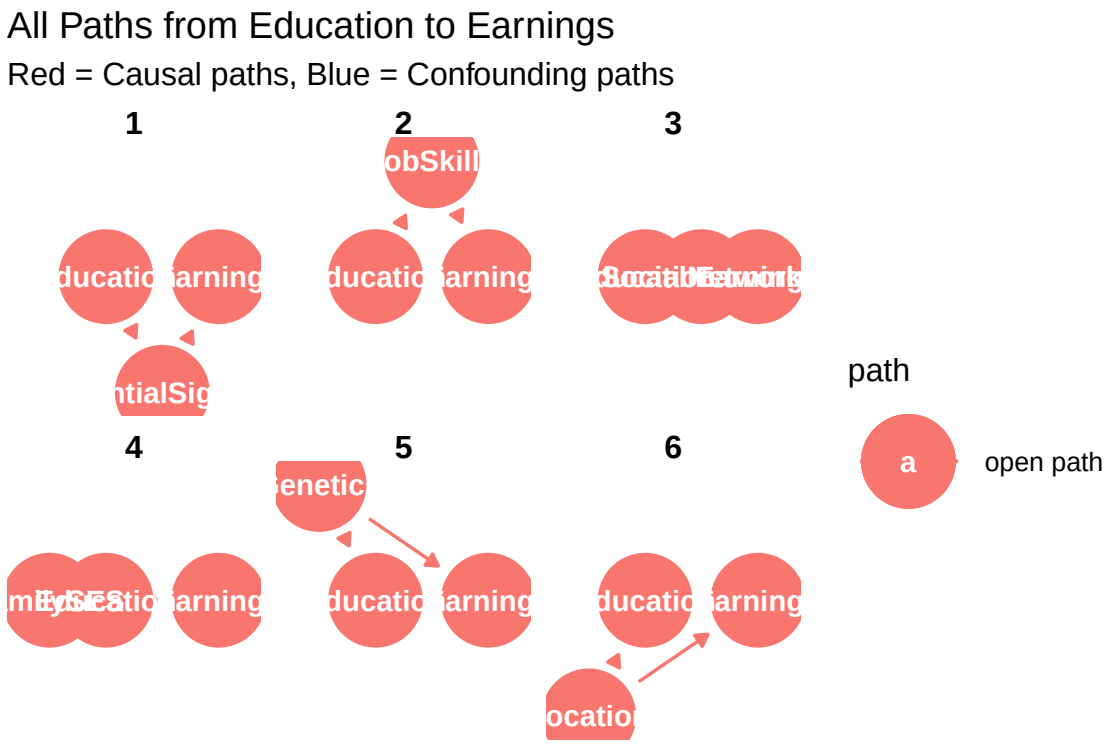


Figure 10: Code for Comprehensive Flow Analysis

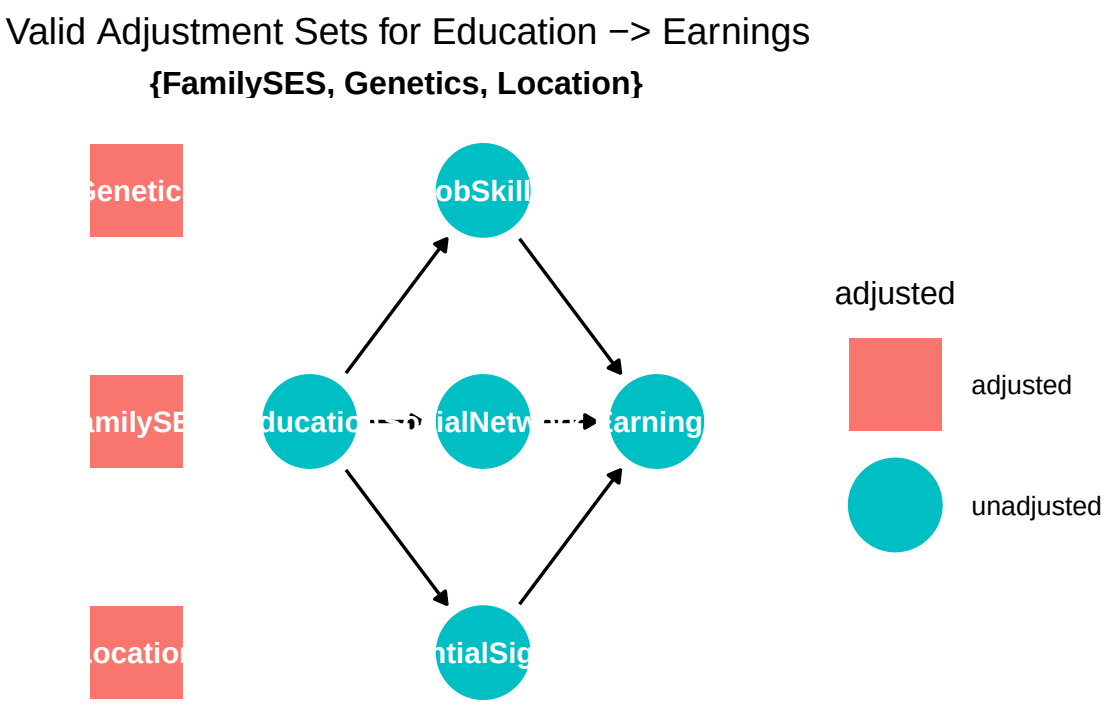


Figure 11: Code for Comprehensive Flow Analysis

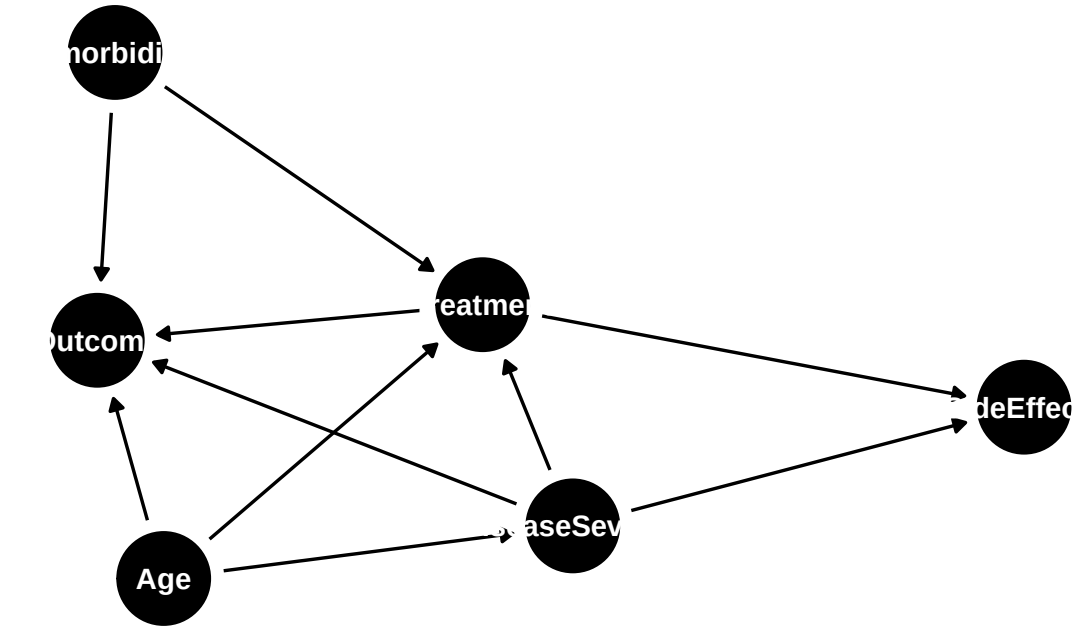


Figure 12: Code for Comprehensive Flow Analysis

> **Code Interpretation**
>
> - **Education example**: Valid adjustment sets block back-door paths while preserving causal mechanisms
> - **Medical example**: Controlling for side effects (collider) biases the treatment effect estimate
> - The simulation shows how collider bias can either inflate or deflate causal effect estimates
> - Real-world lesson: "controlling for everything" can make estimates worse, not better

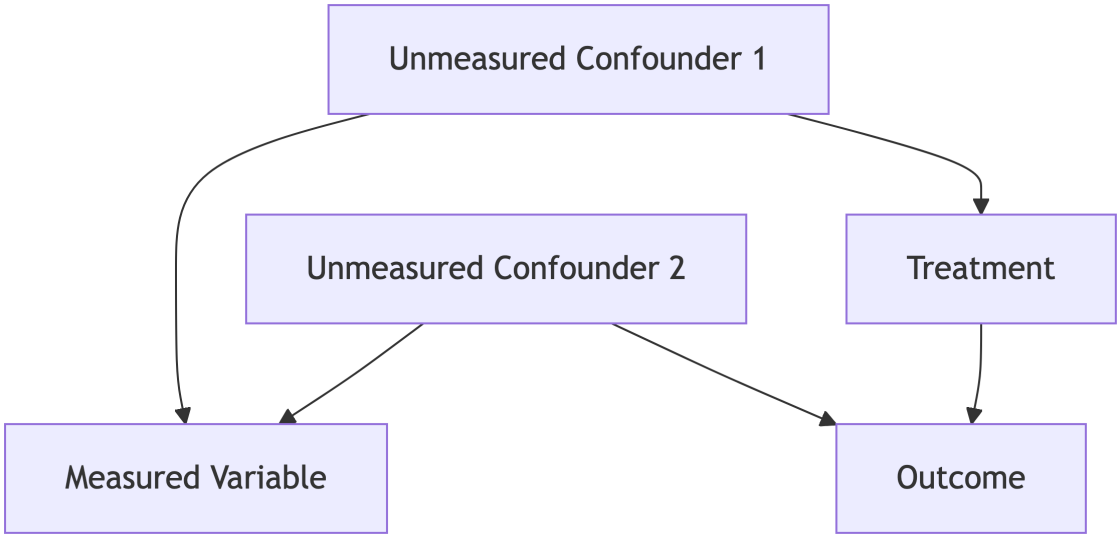
Advanced Topics in Causal Flow

1. M-bias And Complex Collider Structures

Sometimes a variable can simultaneously be a collider and create confounding bias:

```
graph TD
    U1[Unmeasured Confounder 1] --> M[Measured Variable]
    U2[Unmeasured Confounder 2] --> M
```

```
    U1 --> T[Treatment]
    U2 --> O[Outcome]
    T --> O
```

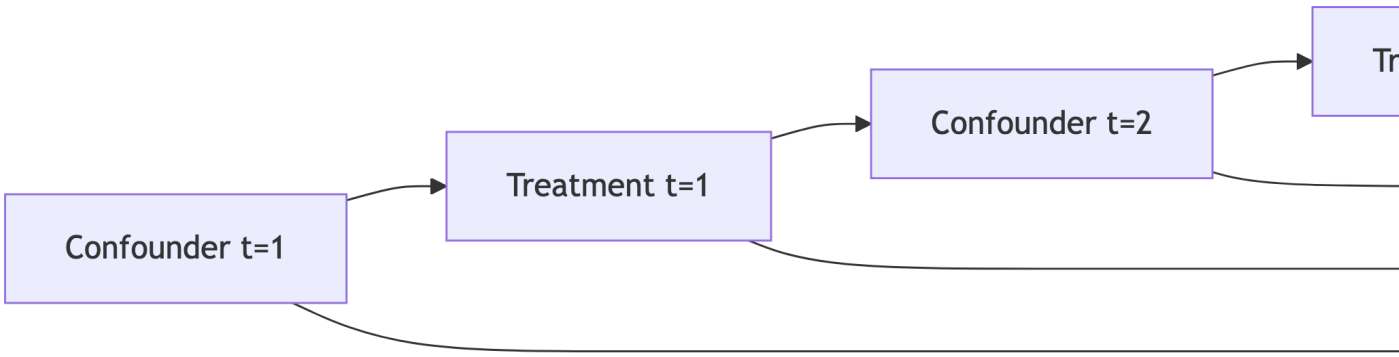


The Dilemma: - M is a collider for U1 and U2 (don't control) - But controlling for M blocks the confounding path T ← U1 → M ← U2 → O - Solution often requires advanced methods (instrumental variables, etc.)

2. Time-varying Confounding and Feedback Loops

In longitudinal studies, past treatments can affect future confounders:

```
graph LR
    L1[Confounder t=1] --> T1[Treatment t=1]
    T1 --> L2[Confounder t=2]
    L2 --> T2[Treatment t=2]
    T1 --> O[Outcome]
    T2 --> O
    L1 --> O
    L2 --> O
```



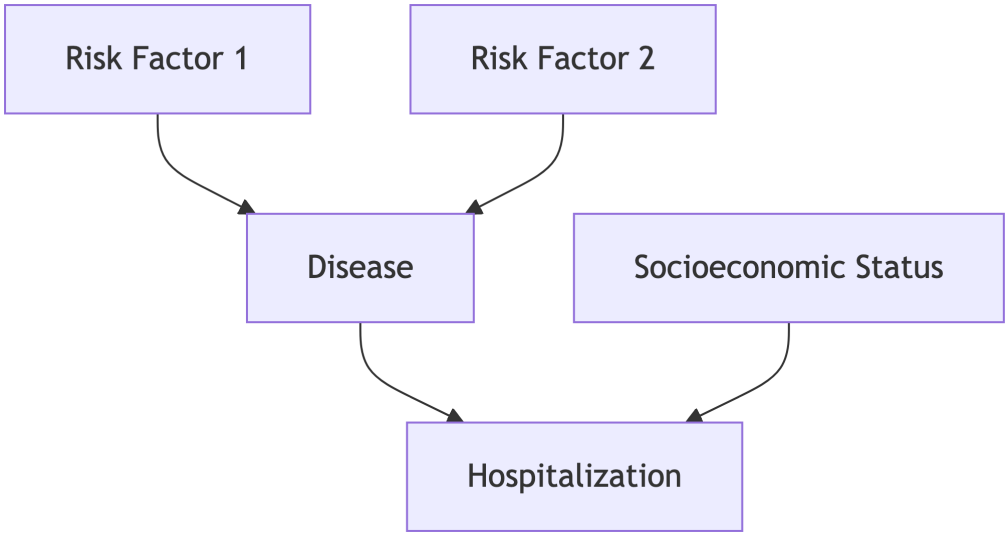
Challenge: L2 is both a confounder (affects T2 and O) and a mediator (affected by T1) **Solution**: Requires specialized methods like g-methods or marginal structural models

3. Selection Bias and Collider Stratification

Many study designs inadvertently condition on colliders:

Examples: - **Hospital-based studies**: Condition on hospitalization - **Volunteer samples**: Condition on willingness to participate
- **Complete-case analysis**: Condition on having complete data - **Survival analysis**: Condition on survival to study entry

```
graph TD
    RF1[Risk Factor 1] --> D[Disease]
    RF2[Risk Factor 2] --> D
    D --> H[Hospitalization]
    SES[Socioeconomic Status] --> H
```

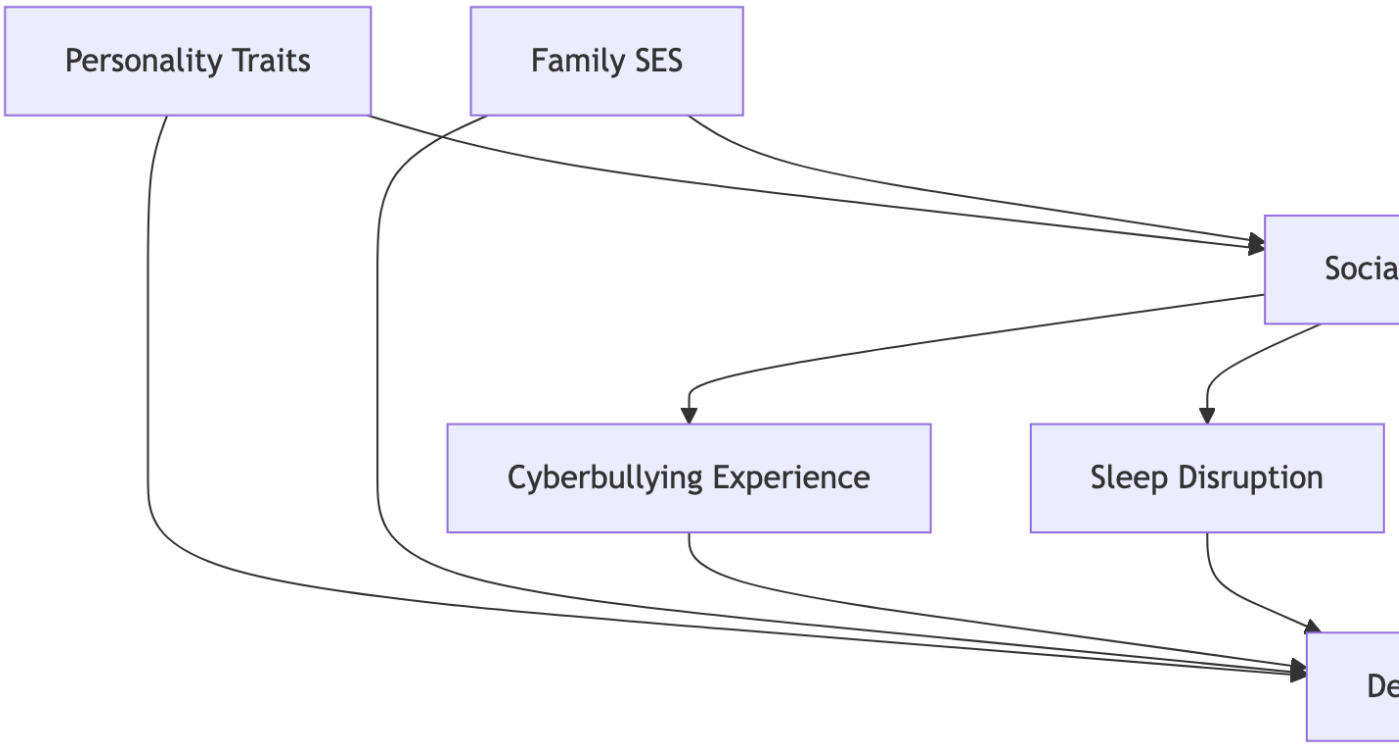


Problem: Hospital studies condition on H (collider), creating spurious associations between RF1 and RF2

Detailed Example 3: Social Media and Mental Health

Research Question: Does social media use cause depression among teenagers?

```
graph TD
    P[Personality Traits] --> SM[Social Media Use]
    P --> D[Depression]
    SES[Family SES] --> SM
    SES --> D
    SM --> Cyber[Cyberbullying Experience]
    Cyber --> D
    SM --> Sleep[Sleep Disruption]
    Sleep --> D
    SM --> FOMO[Fear of Missing Out]
    FOMO --> D
    SM --> Social[Social Comparison]
    Social --> D
    Offline[Offline Social Support] --> SM
    Offline --> D
    School[School Performance] --> SM
    School --> D
```



Causal Pathways (mechanisms through which social media might cause depression): 1. SM → Cyberbullying → Depression 2. SM → Sleep Disruption → Depression
3. SM → FOMO → Depression 4. SM → Social Comparison → Depression

Confounding Pathways: 1. SM ← Personality → Depression (introverted teens use social media more AND are more prone to depression) 2. SM ← Family SES → Depression (SES affects both social media access and mental health resources) 3. SM ← Offline Social Support → Depression (teens with poor offline relationships use social media more AND are depressed)

Analysis Strategy: - **Control for**: Personality, Family SES, Offline Social Support, School Performance - **Don't control for**: Cyberbullying, Sleep Disruption, FOMO, Social Comparison (these are mediators)

R Code: Social Media Analysis

```
# Social Media and Depression DAG
social_media_dag <- dagitty("dag {
  Personality → SocialMediaUse → Cyberbullying → Depression
  Personality → Depression
```

```
  FamilySES → SocialMediaUse → SleepDisruption → Depression
  FamilySES → Depression
  SocialMediaUse → FOMO → Depression
  SocialMediaUse → SocialComparison → Depression
  OfflineSocialSupport → SocialMediaUse
  OfflineSocialSupport → Depression
  SchoolPerformance → SocialMediaUse
  SchoolPerformance → Depression
}")

# Find Adjustment Sets
sm_adjustments <- adjustmentSets(social_media_dag, "SocialMediaUse", "Depression")
cat("=== SOCIAL MEDIA STUDY ADJUSTMENT SETS ===\n")
```

```
=== SOCIAL MEDIA STUDY ADJUSTMENT SETS ===
```

```
print(sm_adjustments)
```

```
{ FamilySES, OfflineSocialSupport, Personality, SchoolPerformance }
```

```
# Test Total Effect Vs Direct Effect Strategies
# Total Effect: Don't Control for Mediators
total_effect_valid <- isAdjustmentSet(social_media_dag, "SocialMediaUse",
"Depression",
                                      c("Personality", "FamilySES",
                                      "OfflineSocialSupport", "SchoolPerformance"))

# Invalid: Controlling for Mediators
controls_mediators <- isAdjustmentSet(social_media_dag, "SocialMediaUse",
"Depression",
                                      c("Personality", "FamilySES", "Cyberbullying",
                                      "SleepDisruption"))

cat("Valid total effect strategy:", total_effect_valid, "\n")
```

```
Valid total effect strategy: FALSE
```

```r
cat("Invalid (controls for mediators):", controls_mediators, "\n")
```

```
Invalid (controls for mediators): FALSE
```

```r
# Visualize the Complex Network
ggdag(social_media_dag) +
  theme_dag_blank() +
  labs(title = "Social Media and Depression: Complex Causal Network",
       subtitle = "Multiple confounders and mediating pathways")
```

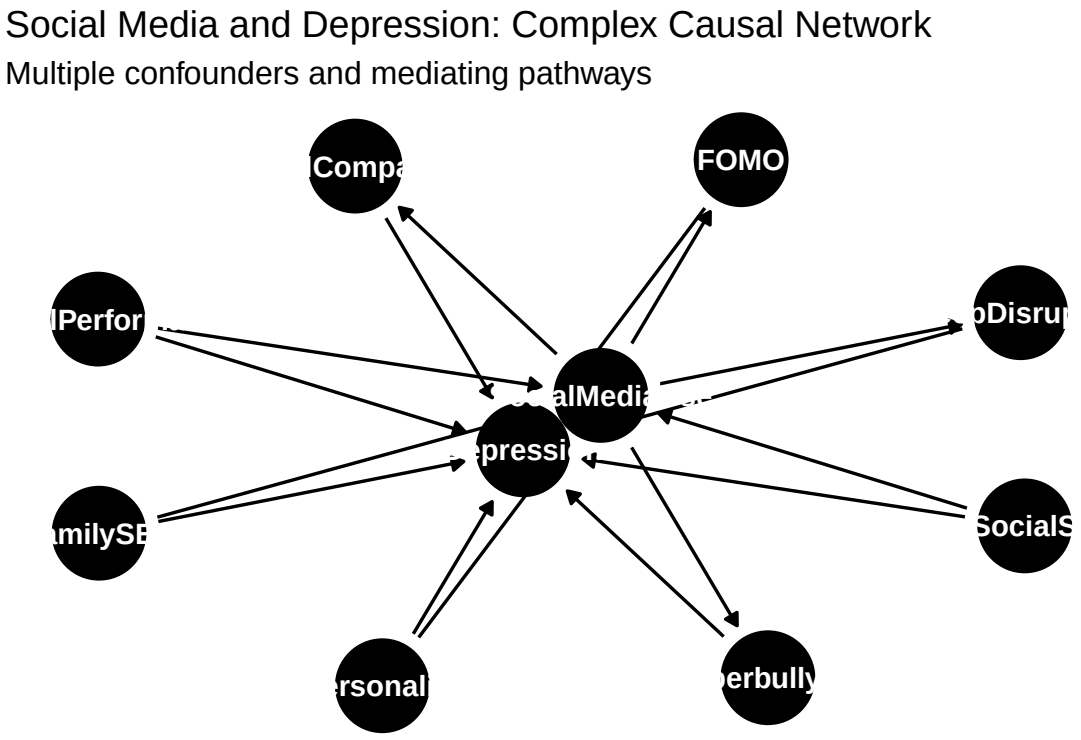


Figure 13: Social Media and Depression DAG

Practical Checklist for Applied Researchers

DAG Construction Checklist

1. **Define your research question clearly**: What is the treatment? What is the outcome?
2. **Brainstorm all relevant variables**: Use subject matter knowledge, not statistical significance
3. **Consider temporal ordering**: Causes must precede effects
4. **Identify potential confounders**: What affects both treatment and outcome?
5. **Think about mechanisms**: How might the treatment affect the outcome?
6. **Look for colliders**: What variables are caused by multiple others?
7. **Consider selection mechanisms**: How was your sample selected?
8. **Test your assumptions**: Are there testable implications of your DAG?

Common Pitfalls to Avoid

1. **The "kitchen sink" approach**: Controlling for everything available
2. **Controlling for mediators**: When you want total effects
3. **Ignoring colliders**: Can create bias when conditioned upon
4. **Post-treatment variables**: Often mediators or colliders
5. **Descendant bias**: Conditioning on descendants of colliders
6. **Reverse causation**: Make sure causal direction is correct
7. **Time-varying confounding**: Static DAGs may be insufficient

Summary: The Complete Framework

The DAG-based approach to causal inference follows this logic:

1. **Theoretical Foundation**: Construct a DAG based on subject matter knowledge
2. **Path Identification**: Identify all causal and confounding paths
3. **d-separation Analysis**: Determine which variables to control for
4. **Assumption Testing**: Check testable implications where possible
5. **Sensitivity Analysis**: Consider robustness to unmeasured confounding

Key Principles Revisited:

- **Chains**: Control for mediators only for direct effects, not total effects
- **Forks**: Always control for common causes (confounders)
- **Colliders**: Don't control unless they're also confounders

- **d-separation**: Use to identify conditional independence relationships
- **Back-door criterion**: Block confounding paths while preserving causal paths

The Ultimate goal: Transform the impossible task of "proving causation from correlation" into the manageable task of "making explicit assumptions and testing their implications."

Check Your Understanding

Question 1: Path Blocking

In the DAG A → B ← C → D, what happens to the association between A and D when you condition on C?

Click for Answer

A and D become associated when you condition on C. The path A → B ← C → D contains a collider at B, but conditioning on C opens the path A → B ← C → D because C is on the causal path from the collider B.

Question 2: Mediation Analysis

Given the chain X → M → Y, what's the difference between controlling and not controlling for M?

Click for Answer

- **Not controlling for M**: Estimates the total effect of X on Y (direct + indirect through M)
- **Controlling for M**: Estimates the direct effect of X on Y (holding M constant)
- **Difference**: The indirect/mediated effect that flows through M

Question 3: Multiple Confounders

In this DAG: Treatment ← Age → Outcome; Treatment ← SES → Outcome; Treatment → Outcome, what should you control for?

Click for Answer

Control for both Age and SES. Both create back-door paths (confounding paths) from Treatment to Outcome. You need to block all back-door paths to get an unbiased estimate of the causal effect.

Question 4: Collider Descendants

If Z is a collider in X → Z ← Y, and W is caused by Z, what happens when you condition on W?

Click for Answer

Conditioning on W (descendant of collider Z) will induce association between X and Y, just like conditioning on Z itself. This is descendant bias - conditioning on any descendant of a collider opens the collider path.

Question 5: Complex Networks

In a study of education on health, you have: SES → Education → Income → Health, and SES → Health. If you want the total effect of education on health, what do you control for?

Click for Answer

Control for SES only. SES is a confounder (back-door path), but Income is a mediator on the causal path Education → Income → Health. Controlling for Income would give you the direct effect, not the total effect.

This comprehensive module provides the theoretical foundation and practical tools needed to understand and apply causal graphs in your research. The next chapter will build on these concepts to explore identification strategies like the back-door and front-door criteria in greater detail.